

Homework – Generazione e Riconoscimento di Tonalità DTMF

Autrice: Elisabetta Roviera

Matricola: s328422

Corso: Analisi tempo-frequenza e Multiscala

Data Esame: 15 settembre 2025

Università: Politecnico di Torino – Ingegneria Matematica

Progetto volto allo studio, generazione e riconoscimento di segnali Dual Tone Multi Frequency (DTMF) attraverso tecniche di sintesi, analisi spettrale e sperimentazioni su vari parametri di acquisizione.

Abstract

MAGARI CAMBIARE ALLA FINE

In questo homework si approfondisce il concetto di **DTMF** (*Dual-Tone Multi-Frequency*), **un sistema di segnali acustici**, tecnologia ampiamente utilizzata nella telefonia per la trasmissione di numeri e simboli mediante combinazioni di tonalità. L'attività prevede la **generazione** e il **riconoscimento** delle tonalità DTMF relative a sequenze numeriche composte, seguite da una serie di **esperimenti mirati** per analizzare l'impatto di parametri quali la durata dei toni, la presenza di rumore, e la frequenza di campionamento. Il lavoro combina strumenti di sintesi e analisi spettrale, con l'obiettivo di consolidare le competenze nella rappresentazione e interpretazione dei segnali in ambito tempo-frequenza.

Indice

1. Esercizio 1 – Creazione del dizionario DTMF
2. Esercizio 2 – Generazione del suono DTMF
3. Esercizio 3 – Generazione della sequenza di toni
4. Esercizio 4 – Analisi spettrale (DFT)
5. Esercizio 5 – Estrazione delle frequenze
6. Esercizio 6 – Valutazione degli effetti sui parametri

Dual-Tone Multi-Frequency DTMF

Dual-Tone Multi-Frequency (DTMF) è una tecnologia utilizzata nelle comunicazioni telefoniche per trasmettere numeri, simboli e comandi tramite una combinazione di due toni audio distinti. Ogni tasto di un telefono, quando premuto, genera una coppia di frequenze specifiche, una ad alta e una a bassa frequenza, che vengono riconosciute dal sistema telefonico per l'identificazione del numero o comando. La combinazione delle frequenze è determinata da una matrice a 4x4, dove le righe rappresentano le frequenze basse (697 Hz, 770 Hz, 852 Hz, 941 Hz) e le colonne quelle alte (1209 Hz, 1336 Hz, 1477 Hz, 1633 Hz). Il sistema DTMF è stato introdotto per sostituire la precedente tecnologia a impulsi, offrendo una trasmissione più rapida e affidabile delle informazioni. Oggi, viene ancora utilizzato nei telefoni fissi, nelle risposte automatiche e in molte applicazioni di telecomunicazione interattiva.

Implementazione DTMF

Import

```
In [1]: from typing import List, Dict, Tuple, Optional

import numpy as np
from scipy.io import wavfile
import matplotlib.pyplot as plt
import os

from copy import deepcopy

from scipy.signal import butter, filtfilt, stft, istft, resample
```

Esercizio 1 – Creazione del dizionario DTMF

Si consideri la tastiera **DTMF** (*Dual-Tone Multi-Frequency*), costituita da una **matrice** con quattro righe e quattro colonne:

- **Ogni riga** corrisponde a una **frequenza bassa** (*low frequency*).
- **Ogni colonna** corrisponde a una **frequenza alta** (*high frequency*).
- Ogni tasto è identificato da una **coppia di frequenze**.

La disposizione tipica è la seguente:

	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

Esempio: Premendo il tasto **1** vengono generate due onde sinusoidali: una a **697 Hz** e una a **1209 Hz**. Il termine *multifrequenza* deriva proprio dall'uso simultaneo di due toni.

a) Siano:

$$F_1 = \{697, 770, 852, 941\}$$

$$F_2 = \{1209, 1336, 1477, 1633\}$$

Creare un **dizionario** `tones` che associ ciascun tasto a una **coppia di indici** (i, j) :

- `i` → indice della frequenza in **F1** (*bassa*).
- `j` → indice della frequenza in **F2** (*alta*).

La struttura è la seguente:

```
tones = {
    '1': (0, 0), '2': (0, 1), '3': (0, 2),
    '4': (1, 0), '5': (1, 1), '6': (1, 2),
    '7': (2, 0), '8': (2, 1), '9': (2, 2),
    '*': (3, 0), '0': (3, 1), '#': (3, 2),
    'A': (0, 3), 'B': (1, 3), 'C': (2, 3), 'D': (3, 3)
}
```

Esempi di corrispondenza:

- **Tasto 1** → `(0, 0)` → Frequenze: `F1[0] = 697 Hz` e `F2[0] = 1209 Hz`.
- **Tasto 5** → `(1, 1)` → Frequenze: `F1[1] = 770 Hz` e `F2[1] = 1336 Hz`.

b) Stampare il dizionario `tones`

```
In [2]: # Esercizio 1: Creazione del dizionario DTMF
# -----
# - definizione degli array F1 (basse) e F2 (alte)
# - costruzione del dizionario `tones` che mappa ogni tasto a (i, j)

# 1- Definizione frequenze
def build_frequency_arrays() -> Tuple[List[int], List[int]]:
    """
    Restituisce le due liste di frequenze:
    F1 (low freq) e F2 (high freq)
    """
    F1 = [697, 770, 852, 941]      # righe (low)
    F2 = [1209, 1336, 1477, 1633] # colonne (high)
    return F1, F2

# 2- Costruzione dizionario tones (a)
def build_tones_dict() -> Dict[str, Tuple[int, int]]:
    """
    Costruisce e restituisce il dizionario `tones` che
    associa ogni tasto alla coppia di indici (i, j).
    """
    tones = {
        '1': (0, 0), '2': (0, 1), '3': (0, 2), 'A': (0, 3),
        '4': (1, 0), '5': (1, 1), '6': (1, 2), 'B': (1, 3),
        '7': (2, 0), '8': (2, 1), '9': (2, 2), 'C': (2, 3),
        '*': (3, 0), '0': (3, 1), '#': (3, 2), 'D': (3, 3)
    }
    return tones

# 3-
def get_tone_indices(key: str, tones: Dict[str, Tuple[int, int]]) -> Tuple[int, int]:
    """
    Restituisce la coppia di indici (i, j) per un tasto.
    Solleva KeyError se il tasto non esiste.
    """
    if key not in tones:
        raise KeyError(f"Tasto '{key}' non valido!")
    return tones[key]

def get_tone_frequencies(key: str,
                        tones: Dict[str, Tuple[int, int]],
                        F1: List[int],
                        F2: List[int]) -> Tuple[int, int]:
    """
    Dato un tasto (es. '1') restituisce le due frequenze (f_low, f_high).
    """
    i, j = get_tone_indices(key, tones)
    return F1[i], F2[j]

# 4- Stampa del dizionario
def print_tones_table(tones: Dict[str, Tuple[int, int]],
                    F1: List[int],
                    F2: List[int]) -> None:
    """
    Stampa il mapping in formato "tastiera" con frequenze associate.
    """
    keypad_rows = [
```

```

    ['1', '2', '3', 'A'],
    ['4', '5', '6', 'B'],
    ['7', '8', '9', 'C'],
    ['*', '0', '#', 'D'],
]

print("Dizionario DTMF (tasto -> (indiceF1, indiceF2) -> (f_low Hz, f_high Hz))")
for row in keypad_rows:
    row_str = []
    for k in row:
        i, j = tones[k]
        f_low, f_high = F1[i], F2[j]
        row_str.append(f"{k}:({i},{j}) -> {f_low}Hz + {f_high}Hz")
    print("    ".join(row_str))

# Stampa del dizionario grezzo
print(f"\nDizionario grezzo (python): {tones}\n")

```

```

In [3]: # ESEMPIO

# Costruisco le frequenze e il dizionario
F1, F2 = build_frequency_arrays()
tones = build_tones_dict()

# (b) Stampare il dizionario
print_tones_table(tones, F1, F2)

# Esempi aggiuntivi per verificare la correttezza del codice:
# come ricavare le frequenze per alcuni tasti
for key in ['1', '5', '0', 'A']:
    f_low, f_high = get_tone_frequencies(key, tones, F1, F2)
    print(f"Tasto '{key}': indici {tones[key]} -> frequenze {f_low} Hz e {f_high} Hz")

```

```

Dizionario DTMF (tasto -> (indiceF1, indiceF2) -> (f_low Hz, f_high Hz))
1:(0,0) -> 697Hz + 1209Hz    2:(0,1) -> 697Hz + 1336Hz    3:(0,2) -> 697Hz + 1477Hz    A:(0,3) -> 697Hz + 1633Hz
4:(1,0) -> 770Hz + 1209Hz    5:(1,1) -> 770Hz + 1336Hz    6:(1,2) -> 770Hz + 1477Hz    B:(1,3) -> 770Hz + 1633Hz
7:(2,0) -> 852Hz + 1209Hz    8:(2,1) -> 852Hz + 1336Hz    9:(2,2) -> 852Hz + 1477Hz    C:(2,3) -> 852Hz + 1633Hz
*(3,0) -> 941Hz + 1209Hz    0:(3,1) -> 941Hz + 1336Hz    #: (3,2) -> 941Hz + 1477Hz    D:(3,3) -> 941Hz + 1633Hz

```

Dizionario grezzo (python): {'1': (0, 0), '2': (0, 1), '3': (0, 2), 'A': (0, 3), '4': (1, 0), '5': (1, 1), '6': (1, 2), 'B': (1, 3), '7': (2, 0), '8': (2, 1), '9': (2, 2), 'C': (2, 3), '*': (3, 0), '0': (3, 1), '#': (3, 2), 'D': (3, 3)}

Tasto '1': indici (0, 0) -> frequenze 697 Hz e 1209 Hz
Tasto '5': indici (1, 1) -> frequenze 770 Hz e 1336 Hz
Tasto '0': indici (3, 1) -> frequenze 941 Hz e 1336 Hz
Tasto 'A': indici (0, 3) -> frequenze 697 Hz e 1633 Hz

Esercizio 2 – Generazione del suono DTMF

Scrivere una funzione `tone` che generi un **segnale sonoro** corrispondente al tasto DTMF specificato, **combinando due onde sinusoidali** di frequenza diversa e restituendo il risultato in un array di valori, pronto per la riproduzione o il salvataggio come file audio.

a) La funzione `tone` deve ricevere in ingresso:

- number** (*string*): il tasto da generare (ad esempio `'1'`, `'2'`, `'*'`, ecc.).
- duration** (*float*): la durata del tono, in secondi.

b) Creare un array di tempi t che rappresenti l'istante di campionamento del segnale, con:

$$t \in [0, \text{duration}], \quad \Delta t = \frac{1}{F_s}$$

dove F_s è la **frequenza di campionamento**.

c) Calcolare il segnale:

$$x(t) = \cos(2\pi f_1 t) + \cos(2\pi f_2 t)$$

dove f_1 e f_2 sono le frequenze **bassa** e **alta** associate al tasto DTMF scelto.

d) Dopo aver generato il segnale:

- Visualizzare** il segnale nel dominio del tempo.
- Salvare** il segnale in un file audio.
- Riprodurre** il suono generato.

```

In [4]: # Esercizio 2 - Generazione del suono DTMF
# -----
# - Definizione funzione tone
# - Creazione dell'array dei tempi
# - Generazione del segnale x(t) = cos(2π f1 t) + cos(2π f2 t)
# - Plot del segnale (tempo)
# - Salvataggio in WAV (mono, int16)

# 1- Funzione tone(number, duration, fs, amplitude)
def tone(number: str,
          duration: float,
          fs: int = 8000,
          amplitude: float = 0.8,

```

```

    ) -> Tuple[np.ndarray, np.ndarray, int]:
    """
    Genera il segnale DTMF per il tasto `number`.

    Parametri:
    - number: tasto come stringa, es. '1' o '*'
    - duration: durata in secondi (float)
    - fs: frequenza di campionamento (default 8000 Hz)
    - amplitude: ampiezza massima normalizzata (0..1)

    Restituisce: (t, x, fs)
    - t: array dei tempi
    - x: segnale normalizzato in float (-1..1)
    - fs: sample rate
    """
    # Preleva mappe e frequenze
    F1, F2 = build_frequency_arrays()
    tones = build_tones_dict()
    f_low, f_high = get_tone_frequencies(number, tones, F1, F2)

    # Vettore dei tempi
    t = np.arange(0.0, duration, 1.0 / fs)

    # Somma dei due coseni
    x = np.cos(2 * np.pi * f_low * t) + np.cos(2 * np.pi * f_high * t)

    # Normalizzazione e scala all'ampiezza richiesta
    max_val = np.max(np.abs(x))
    if max_val > 0:
        x = (x / max_val) * amplitude

    return t, x, fs

# 2- Plotting del segnale
def plot_signal(t: np.ndarray, x: np.ndarray, title: str, max_points: int = None):
    """
    Plotta il segnale nel dominio del tempo.
    Se max_points è fornito mostra solo i primi max_points campioni.
    """
    plt.figure(figsize=(10, 3.5))
    if max_points is not None:
        plt.plot(t[:max_points], x[:max_points])
    else:
        plt.plot(t, x)
    plt.title(title)
    plt.xlabel("Tempo [s]")
    plt.ylabel("Ampiezza")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Funzione per plottare i tasti in una griglia 4x4
def plot_tones_grid(tones: Dict[str, Tuple[int, int]], F1: List[int], F2: List[int], duration: float = 0.09, fs: int = 8000, amp: float = 0.8,
                    fig, axs = plt.subplots(4, 4, figsize=(15, 12)) # 4 righe x 4 colonne):
    # Mappa per la tastiera DTMF
    keypad_rows = [
        ['1', '2', '3', 'A'],
        ['4', '5', '6', 'B'],
        ['7', '8', '9', 'C'],
        ['*', '0', '#', 'D'],
    ]

    # Cicla sui tasti DTMF e plottali nella griglia
    for i, row in enumerate(keypad_rows):
        for j, key in enumerate(row):
            # Genera il tono per il tasto
            t, x, fs_out = tone(key, duration, fs=fs, amplitude=amp)

            # Seleziona il subplot corrispondente
            ax = axs[i, j]
            ax.plot(t, x)
            ax.set_title(f"Tasto '{key}'")
            ax.set_xlabel("Tempo [s]")
            ax.set_ylabel("Ampiezza")
            ax.grid(True)

    plt.tight_layout()
    plt.show()

# 3- Funzione per salvare in WAV
def save_wav(filename: str, x: np.ndarray, fs: int, out_dir: str = ".") -> str:
    """
    Salva il segnale mono x (float -1..1) in un file WAV 16-bit.
    Restituisce il percorso del file salvato.
    """
    # Crea cartella se necessaria
    os.makedirs(out_dir, exist_ok=True)

    # Scala su int16
    int_data = np.int16(np.clip(x, -1.0, 1.0) * 32767)
    out_path = os.path.join(out_dir, filename)

```

```
wavfile.write(out_path, fs, int_data)
return out_path
```

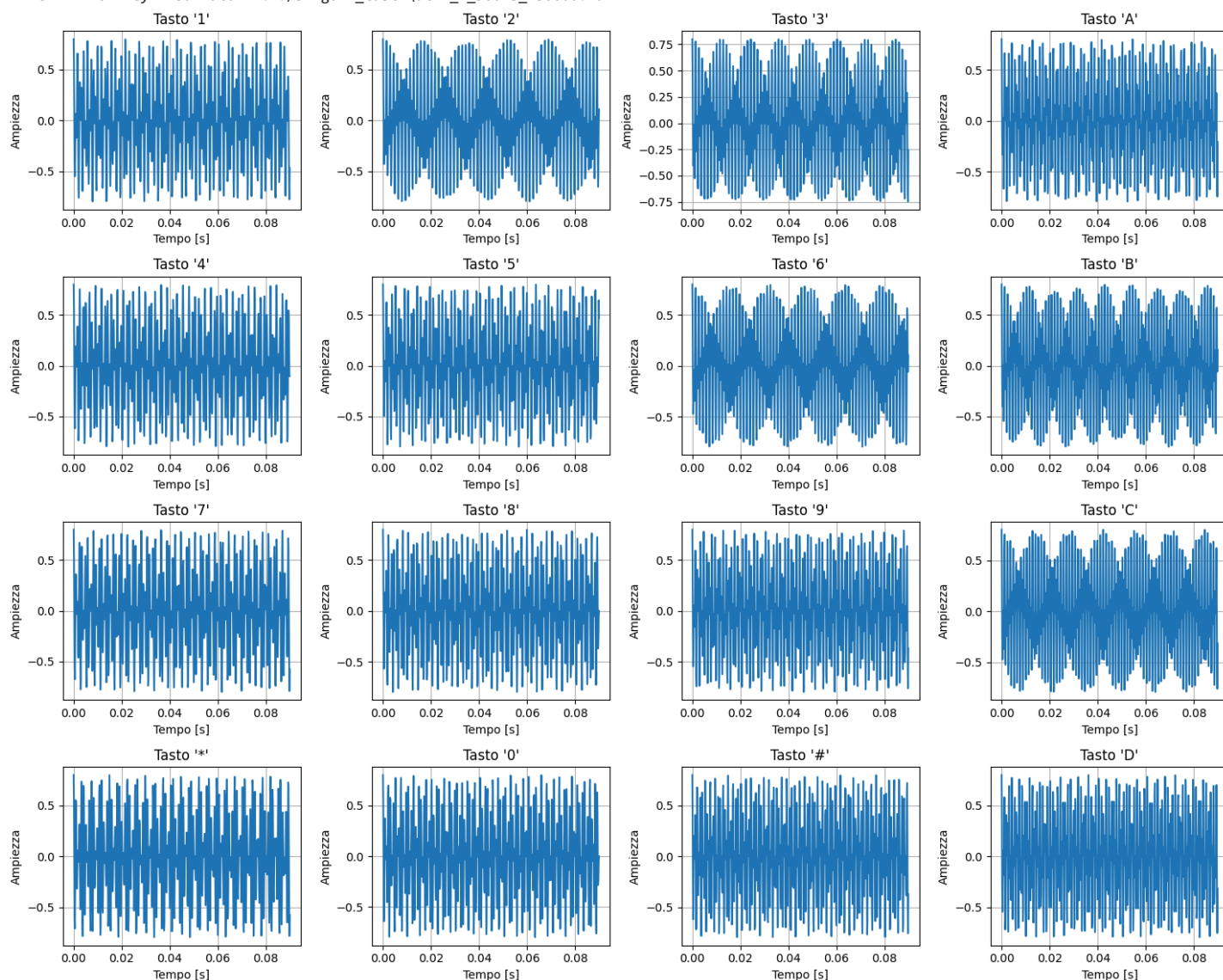
```
In [5]: # ESEMPIO
# Parametri di esempio
for key in tones.keys():
    duration = 0.5 # secondi
    fs = 8000 # campionamento
    amp = 0.8 # ampiezza normalizzata
    out_folder = "./singoli_tasti" # cartella di salvataggio del WAV

    # Genera il tono
    t, x, fs_out = tone(key, duration, fs=fs, amplitude=amp)

    # Salvataggio WAV (file mono, int16)
    filename = f"dtmf_{key.replace('*', '_')}_{int(duration*1000)}ms_fs{fs_out}.wav"
    path = save_wav(filename, x, fs_out, out_dir=out_folder)
    print(f"File WAV for key {key} salvato in: {path}")

if key == 'D':
    # Chiamata alla funzione di plot per tutti i tasti
    plot_tones_grid(tones, F1, F2)
```

```
File WAV for key 1 salvato in: ./singoli_tasti\dtmf_1_500ms_fs8000.wav
File WAV for key 2 salvato in: ./singoli_tasti\dtmf_2_500ms_fs8000.wav
File WAV for key 3 salvato in: ./singoli_tasti\dtmf_3_500ms_fs8000.wav
File WAV for key A salvato in: ./singoli_tasti\dtmf_A_500ms_fs8000.wav
File WAV for key 4 salvato in: ./singoli_tasti\dtmf_4_500ms_fs8000.wav
File WAV for key 5 salvato in: ./singoli_tasti\dtmf_5_500ms_fs8000.wav
File WAV for key 6 salvato in: ./singoli_tasti\dtmf_6_500ms_fs8000.wav
File WAV for key B salvato in: ./singoli_tasti\dtmf_B_500ms_fs8000.wav
File WAV for key 7 salvato in: ./singoli_tasti\dtmf_7_500ms_fs8000.wav
File WAV for key 8 salvato in: ./singoli_tasti\dtmf_8_500ms_fs8000.wav
File WAV for key 9 salvato in: ./singoli_tasti\dtmf_9_500ms_fs8000.wav
File WAV for key C salvato in: ./singoli_tasti\dtmf_C_500ms_fs8000.wav
File WAV for key * salvato in: ./singoli_tasti\dtmf_500ms_fs8000.wav
File WAV for key 0 salvato in: ./singoli_tasti\dtmf_0_500ms_fs8000.wav
File WAV for key # salvato in: ./singoli_tasti\dtmf_#_500ms_fs8000.wav
File WAV for key D salvato in: ./singoli_tasti\dtmf_D_500ms_fs8000.wav
```



Commento sul Grafico.

Il grafico mostra i segnali in **dominio del tempo** per ciascun tasto della tastiera DTMF, generato secondo il codice DTMF. Ogni grafico rappresenta il tono del tasto corrispondente, dove ogni tasto è identificato da un numero (1-9), simboli (*, #) e lettere (A, B, C, D), che sono organizzati in una matrice di 4 righe e 4 colonne.

1. Distribuzione delle frequenze:

- I segnali sono generati dalla somma di due onde sinusoidali, una di bassa frequenza (fissata da `F1`, le righe della tastiera) e una di alta frequenza (definita da `F2`, le colonne della tastiera). Ogni tasto è quindi rappresentato dalla combinazione di due frequenze, che si riflettono nelle forme d'onda sinusoidali visibili in ciascun grafico.

2. Struttura della tastiera DTMF:

- La **prima riga** contiene i tasti '1', '2', '3', e 'A'. La **seconda riga** contiene '4', '5', '6', e 'B', e così via fino alla **quarta riga** con i tasti '*', '0', '#', e 'D'.
- Ogni tasto è rappresentato da una forma d'onda che mostra la combinazione delle due frequenze corrispondenti. Per esempio, il tasto '1' è formato dalle frequenze 697 Hz (bassa) e 1209 Hz (alta).

3. Durata e ampiezza:

- Tutti i segnali hanno una **durata di 0,5 secondi**, come definito nel codice. Questo è visibile nei grafici, dove ogni onda sinusoidale si ripete per un intervallo di tempo di mezzo secondo.
- Le ampiezze sono normalizzate, e variano tra -1 e 1, come evidenziato dal range verticale delle onde.

4. Caratteristiche delle onde:

- Le onde generate per ogni tasto mostrano un comportamento tipico di **onde sinusoidali**. Le oscillazioni risultano **periodiche e regolari**, corrispondenti ai toni puri delle frequenze di ciascun tasto.
- La forma d'onda può sembrare "piena" o più "frastagliata", ma ogni tasto ha una combinazione unica di frequenze che crea una somma di onde sinusoidali.

Questa visualizzazione fornisce una panoramica chiara delle onde DTMF associate ai tasti di una tastiera telefonica. I segnali sono generati in base alla combinazione di due frequenze distintive per ciascun tasto, e la griglia 4x4 facilita la comparazione tra i diversi segnali. L'uso di frequenze sinusoidali e la loro durata costante permette di identificare facilmente ogni tasto in un sistema DTMF, sia visivamente che sonoramente.

La normalizzazione dell'ampiezza permette che tutte le onde abbiano un'ampiezza coerente, e la durata di 0,5 secondi è un intervallo adeguato per ogni tasto, rendendo ogni segnale chiaramente riconoscibile durante l'analisi.

Esercizio 3 – Generazione della sequenza di toni

a) Scrivere una funzione `dialNumber` che prenda due argomenti:

- `numbers`: una stringa di numeri o simboli, che rappresenta i tasti da "premere" (ad esempio, un numero di telefono).
- `toneDuration`: la durata di ciascun tono in secondi.

b) Generare la lista di toni, uno per ogni cifra o simbolo nella stringa `numbers` e generare il segnale audio corrispondente per la durata specificata in `toneDuration` usando la funzione precedentemente definita.

c) Concatenare i toni e restituire una sequenza continua di dati audio che rappresenta l'intera sequenza di tasti premuti.

d) Visualizzare poi il segnale nel tempo e salvare il file audio.

```
In [6]: # Esercizio 3 - Generazione della sequenza di toni DTMF
# -----
# - Definizione di dialNumber: genera sequenza concatenata di toni DTMF dato un input string
# - Definizione di numbers: stringa di tasti (es. "123*A")
# - Plotting del segnale risultante e salvataggio in WAV

# 1- Funzione dialNumber
def dialNumber(numbers: str,
               toneDuration: float,
               fs: int = 8000,
               interToneGap: float = 0.05,
               amplitude: float = 0.8,
               out_dir: str = "./",
               filename_prefix: str = None,
               plot: bool = False) -> Tuple[np.ndarray, int, str]:
    """
    Genera la sequenza concatenata di toni DTMF per la stringa `numbers`.

    Parametri:
    - numbers: stringa di tasti (es. "123*A")
    - toneDuration: durata di ciascun tono in secondi
    - fs: frequenza di campionamento
    - interToneGap: pausa di silenzio tra i toni (secondi)
    - amplitude: ampiezza normalizzata dei toni
    - out_dir: cartella di salvataggio del WAV
    - filename_prefix: prefisso del file WAV (default: dtmf_sequence_<numbers>)

    Restituisce:
    - x: array concatenato dei dati audio (float -1..1)
    - fs: sample rate
    - path: percorso del file WAV salvato
    """
    # validazioni di base
    if toneDuration <= 0:
        raise ValueError("toneDuration deve essere > 0.")
    if interToneGap < 0:
        raise ValueError("interToneGap non può essere negativo.")

    tones = build_tones_dict() # per validare i caratteri
    # costruisco il vettore di silenzio per la pausa (se interToneGap > 0)
    gap = np.zeros(int(np.round(interToneGap * fs)), dtype=float) if interToneGap > 0 else np.array([], dtype=float)
```

```

sequence_parts = []
time_offsets = [] # per sapere dove inizia ogni tono (in secondi)
current_time = 0.0

for idx, ch in enumerate(numbers):
    if ch not in tones:
        raise KeyError(f"Carattere '{ch}' non valido per DTMF.")
    # genera il tono per il singolo tasto
    t, x_tone, _ = tone(ch, toneDuration, fs=fs, amplitude=amplitude)
    sequence_parts.append(x_tone)
    time_offsets.append(current_time)
    current_time += len(x_tone) / fs

    # aggiungi gap (tranne dopo l'ultimo tono)
    if idx < len(numbers) - 1 and gap.size > 0:
        sequence_parts.append(gap)
        current_time += gap.size / fs

if len(sequence_parts) == 0:
    x = np.array([], dtype=float)
    total_time = 0.0
else:
    x = np.concatenate(sequence_parts)
    total_time = len(x) / fs

# plotting: segnale completo (attenzione per sequenze molto lunghe)
if plot == True:
    t_full = np.arange(0, len(x)) / fs
    plot_signal(t_full, x, f"Sequenza DTMF '{numbers}' - durata totale {total_time:.3f} s")

# salva in WAV
safe_numbers = "".join(ch if ch.isalnum() else {'*': 'star', '#': 'hash'}.get(ch, ch) for ch in numbers)
if filename_prefix is None:
    filename_prefix = f"dtmf_sequence_{safe_numbers}_{int(toneDuration*1000)}ms"
filename = f"{filename_prefix}_fs{fs}.wav"
path = save_wav(filename, x, fs, out_dir=out_dir)

print(f"File WAV salvato in: {path}")
return x, fs, path

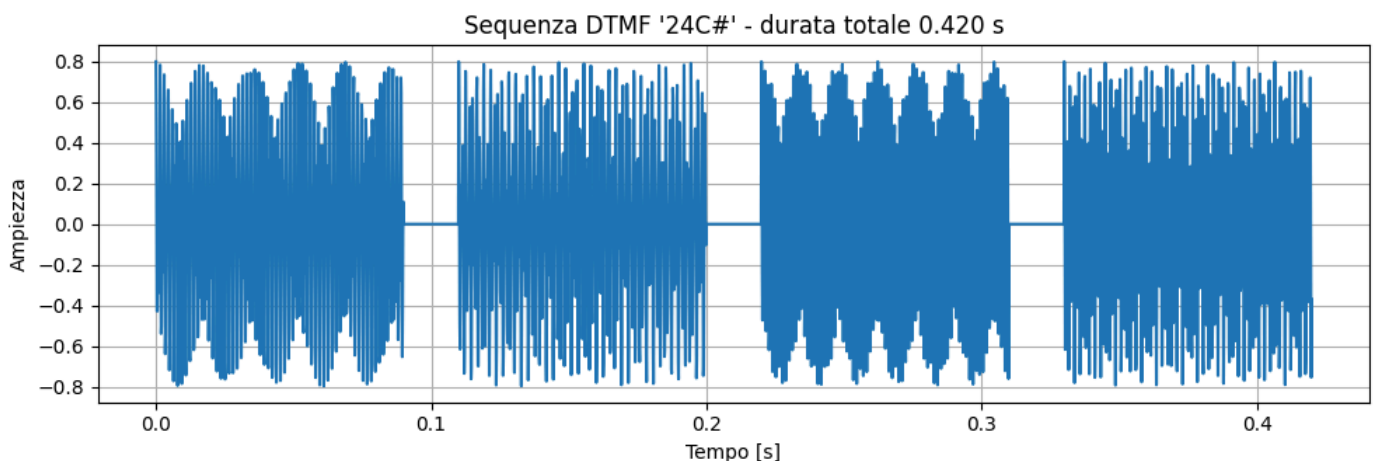
```

```

In [7]: # ESEMPI
numbers = "24C#"
toneDuration = 0.09 # secondi per tono
fs = 8000
interToneGap = 0.02 # 50 ms tra toni
out_dir = "./sequenza_tasti" # cartella di salvataggio

x, fs_out, wav_path = dialNumber(numbers, toneDuration, fs=fs,
                                  interToneGap=interToneGap,
                                  amplitude=0.8,
                                  out_dir=out_dir,
                                  plot = True)

```



File WAV salvato in: ./sequenza_tasti\dtmf_sequence_24Chash_90ms_fs8000.wav

Commento sul Grafico.

Il grafico mostra l'**andamento nel tempo** della sequenza di toni DTMF per la stringa di tasti "24C#". Ogni parte della sequenza corrisponde ai toni generati per un tasto specifico.

1. Composizione della Sequenza:

- Il grafico è diviso in **4 sezioni**, ognuna delle quali corrisponde al suono di un tasto DTMF:
 - Il **primo segmento** è il tono per il tasto '2'.
 - Il **secondo segmento** è il tono per il tasto '4'.
 - Il **terzo segmento** è il tono per il tasto 'C'.
 - Il **quarto segmento** è il tono per il tasto '#'.
- Ogni tono è generato dalla somma di due onde sinusoidali a frequenze diverse, una bassa e una alta.

2. Durata e Ampiezza:

- La durata totale della sequenza è di **0.420 secondi**, come indicato nel titolo del grafico.
- I **toni** sono generati con un'**ampiezza normalizzata** che varia tra -1 e 1, come si vede dal range verticale del grafico.
- Ogni tono ha una durata di **0.09 secondi**, corrispondente alla `toneDuration` definita nel codice.

3. Pausa tra i Toni:

- Nel grafico si possono notare piccole **pause** tra ogni sezione, corrispondenti al gap tra i toni. La durata di queste pause è di **0.02 secondi** (20 ms), come specificato dal parametro `interToneGap`.
- Le pause non sono visibili come vere e proprie aree di "silenzio", ma si percepiscono come l'assenza di segnali tra le onde dei toni successivi.

4. Forma d'onda:

- Ogni **segmento di onda** rappresenta un tono sinusoidale corrispondente a un tasto DTMF.
- Le forme d'onda mostrano un **pattern regolare e periodico** che è tipico delle onde sinusoidali, con oscillazioni corrispondenti alle frequenze basse e alte associate a ciascun tasto.

5. Comportamento dei Tasti:

- La sequenza inizia con il tasto `'2'`, seguito dal tasto `'4'`, quindi la lettera `'C'` e infine il simbolo `'#'`. Ogni tasto è rappresentato da una combinazione unica di frequenze, il che si riflette nelle diverse forme d'onda per ciascun segmento del grafico.

Questo grafico mostra chiaramente come ogni tasto venga rappresentato da una forma d'onda sinusoidale nel dominio del tempo, con pause tra di essi. Ogni tasto, che può essere un numero, un simbolo o una lettera, è distintamente riconoscibile grazie alla sua frequenza unica. La visualizzazione è utile per comprendere come la sequenza DTMF venga generata e come i tasti vengano separati tra loro.

Esercizio 4 – Analisi spettrale (DFT)

Generare un array da $-Fs/2$ a $Fs/2$, dove Fs è la frequenza di campionamento con un numero di punti pari alla lunghezza del segnale audio. Calcolare i coefficienti della DFT e visualizzarne il modulo nel dominio delle frequenze. È possibile identificare le frequenze dei toni?

```
In [8]: # Esercizio 4 - Analisi spettrale (DFT)
# -----
# - Computazione della DFT (con fftshift) su un segnale audio
# - Costruzione dell'asse delle frequenze da -Fs/2 a Fs/2
# - Visualizzazione del modulo (Lineare e/o in dB)

# 1- Computazione della DFT del segnale
def compute_spectrum(x: np.ndarray, fs: int, zero_pad: int = 1) -> Tuple[np.ndarray, np.ndarray]:
    """
    Calcola la DFT del segnale x e ritorna il modulo dello spettro centrato.
    Parametri:
    - x: segnale audio mono (1D)
    - fs: frequenza di campionamento
    - zero_pad: fattore di zero-padding (1 = nessun padding; 4 = N->4N)
    Restituisce:
    - f_shifted: asse delle frequenze centrato in zero (da -fs/2 a +fs/2)
    - Xs_mag: modulo dello spettro corrispondente (stessa lunghezza di f_shifted)
    """
    N = len(x)
    w = np.ones(N) # finestra rettangolare
    xw = x * w

    # Zero-padding (aggiunta di zeri per migliorare la risoluzione spettrale)
    Npad = int(N * zero_pad)
    X = np.fft.fft(xw, n=Npad)
    Xs = np.fft.fftshift(X) # Shift dello spettro per centrarlo a zero
    Xs_mag = np.abs(Xs) # Modulo del segnale spettrale

    # Creazione dell'asse delle frequenze centrato
    f_shifted = np.linspace(-fs/2, fs/2, len(Xs_mag), endpoint=False)

    return f_shifted, Xs_mag

# Funzione per plottare lo spettro in modulo lineare e in dB
def plot_spectrum(f: np.ndarray, X_mag: np.ndarray, title: str = "Spettro (modulo)", show_db: bool = True, xlim: Tuple[float, float] = None):
    """
    Plotta lo spettro in modulo lineare e in dB.
    Parametri:
    - f: asse delle frequenze (Hz), centrato in zero
    - X_mag: modulo dello spettro
    - show_db: se True mostra anche il plot in dB sotto
    - xlim: opzione per zoomare su una fascia di frequenze specifica (es. (min, max))
    """
    if show_db:
        # Due subplots verticali: modulo lineare sopra e modulo in dB sotto
        fig, axs = plt.subplots(2, 1, figsize=(10, 6), sharex=True, gridspec_kw={'height_ratios': [2, 1]})
        axs[0].plot(f, X_mag)
        axs[0].set_title(title + " - modulo lineare")
        axs[0].set_ylabel("Ampiezza")
        axs[0].grid(True)
        if xlim is not None:
            axs[0].set_xlim(xlim)

        X_db = 20 * np.log10(X_mag + 1e-12) # In dB, evitando Log(0)
        axs[1].plot(f, X_db)
        axs[1].set_title(title + " - modulo in dB")
        axs[1].set_xlabel("Frequenza [Hz]")
```



```

        axs[1].set_ylabel("Ampiezza [dB]")
        axs[1].grid(True)
        if xlim is not None:
            axs[1].set_xlim(xlim)

        plt.tight_layout()
        plt.show()
    else:
        plt.figure(figsize=(10, 4))
        plt.plot(f, X_mag)
        plt.title(title + " - modulo lineare")
        plt.xlabel("Frequenza [Hz]")
        plt.ylabel("Ampiezza")
        if xlim is not None:
            plt.xlim(xlim)
        plt.grid(True)
        plt.tight_layout()
        plt.show()

# Funzione per misurare i picchi locali intorno a frequenze candidate
def measure_candidates(f: np.ndarray, X_mag: np.ndarray, candidates: List[float], delta_hz: float = 5.0) -> Dict[float, Tuple[float, float]]:
    """
    Misura il picco locale vicino a ciascuna frequenza candidata.
    Parametri:
        - f: array delle frequenze calcolato dalla FFT
        - X_mag: modulo dello spettro
        - candidates: frequenze candidate da cercare (ad esempio, F1 o F2)
        - delta_hz: tolleranza in Hz per la finestra di ricerca
    Restituisce:
        - Un dizionario con la frequenza candidata come chiave e il massimo locale come valore
    """
    out = {}
    for cand in candidates:
        idxs = np.where((f >= cand - delta_hz) & (f <= cand + delta_hz))[0] # Finestra di ricerca
        if idxs.size == 0:
            out[cand] = (np.nan, 0.0)
            continue
        sub_mag = X_mag[idxs]
        k = np.argmax(sub_mag) # Indice del massimo nella finestra
        best_idx = idxs[k]
        out[cand] = (f[best_idx], X_mag[best_idx])
    return out

# Funzione per identificare le frequenze DTMF (basse e alte)
def identify_dtmf_frequencies(x: np.ndarray, fs: int, F1: List[float], F2: List[float], zero_pad: int = 1, delta_hz: float = 8.0, plot: bool)
    """
    Calcola lo spettro del segnale e misura le frequenze DTMF per le bande basse (F1) e alte (F2).
    Parametri:
        - x: segnale audio DTMF
        - fs: frequenza di campionamento
        - F1: frequenze basse DTMF
        - F2: frequenze alte DTMF
        - zero_pad: fattore di zero-padding
        - delta_hz: tolleranza per l'identificazione dei picchi
        - plot: se True, visualizza lo spettro e i picchi
    Restituisce:
        - Un dizionario con le misure per le bande F1 e F2
    """
    # Calcola lo spettro
    f, X_mag = compute_spectrum(x, fs, zero_pad=zero_pad)

    # Misura i picchi per le frequenze basse (F1) e alte (F2)
    meas_F1 = measure_candidates(f, X_mag, F1, delta_hz=delta_hz)
    meas_F2 = measure_candidates(f, X_mag, F2, delta_hz=delta_hz)

    if plot:
        # Visualizza lo spettro completo con modulo lineare e in dB
        plot_spectrum(f, X_mag, title="Spettro DTMF", show_db=True)

    return {'F1': meas_F1, 'F2': meas_F2}

# Plotta il segnale audio nel dominio del tempo e il suo spettro nel dominio delle frequenze
def plot_signal_and_spectrum(x: np.ndarray, fs: int, zero_pad: int = 1):
    """
    Plotta il segnale audio nel dominio del tempo e il suo spettro nel dominio delle frequenze.
    """
    # Calcolo dello spettro
    f, X_mag = compute_spectrum(x, fs, zero_pad=zero_pad)

    # Creazione della figura con due sottotitoli (side by side)
    fig, axs = plt.subplots(1, 2, figsize=(14, 6))

    # Plot del segnale nel dominio del tempo (prima colonna)
    axs[0].plot(np.arange(len(x)) / fs, x)
    axs[0].set_title("Segnale nel dominio del tempo")
    axs[0].set_xlabel("Tempo [s]")
    axs[0].set_ylabel("Ampiezza")
    axs[0].grid(True)

    # Plot dello spettro nel dominio della frequenza (seconda colonna)
    axs[1].plot(f, X_mag)
    axs[1].set_title("Spettro nel dominio delle frequenze")
    axs[1].set_xlabel("Frequenza [Hz]")

```

```

    axs[1].set_ylabel("Ampiezza")
    axs[1].grid(True)

    plt.tight_layout()
    plt.show()

```

```

In [9]: # ESEMPIO
# Genera il segnale DTMF per una sequenza di tasti
numbers = "24C#"
toneDuration = 0.09 # secondi per tono
fs = 8000
interToneGap = 0.02 # 50 ms tra toni
out_dir = "./analisi_spettrale" # cartella di salvataggio

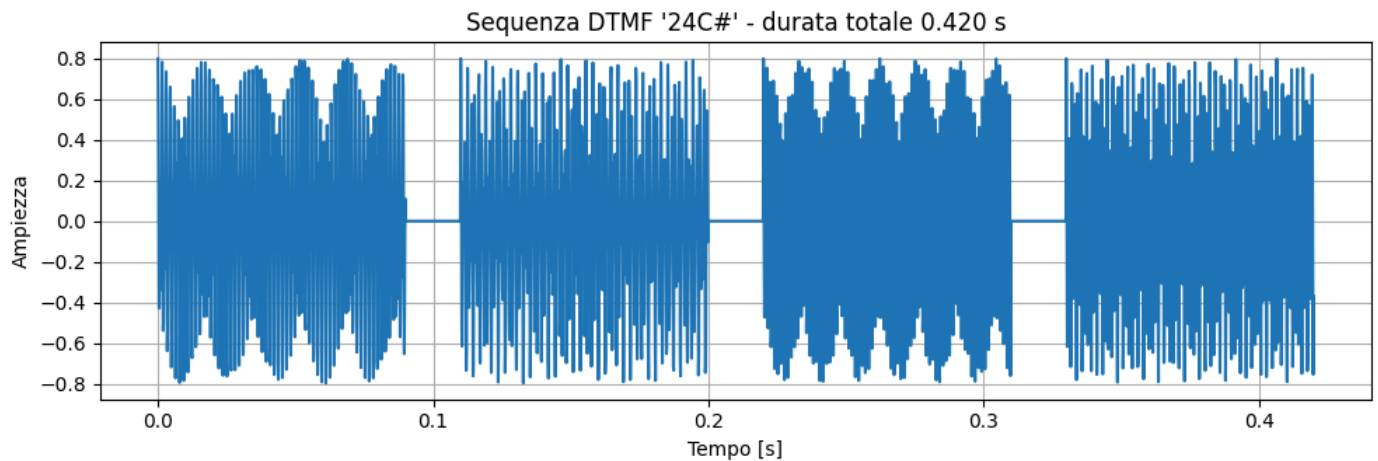
x, fs_out, wav_path = dialNumber(numbers, toneDuration, fs=fs,
                                interToneGap=interToneGap,
                                amplitude=0.8,
                                out_dir=out_dir,
                                plot = True)

# Frequenze basse (F1) e alte (F2) per DTMF
F1, F2 = build_frequency_arrays()

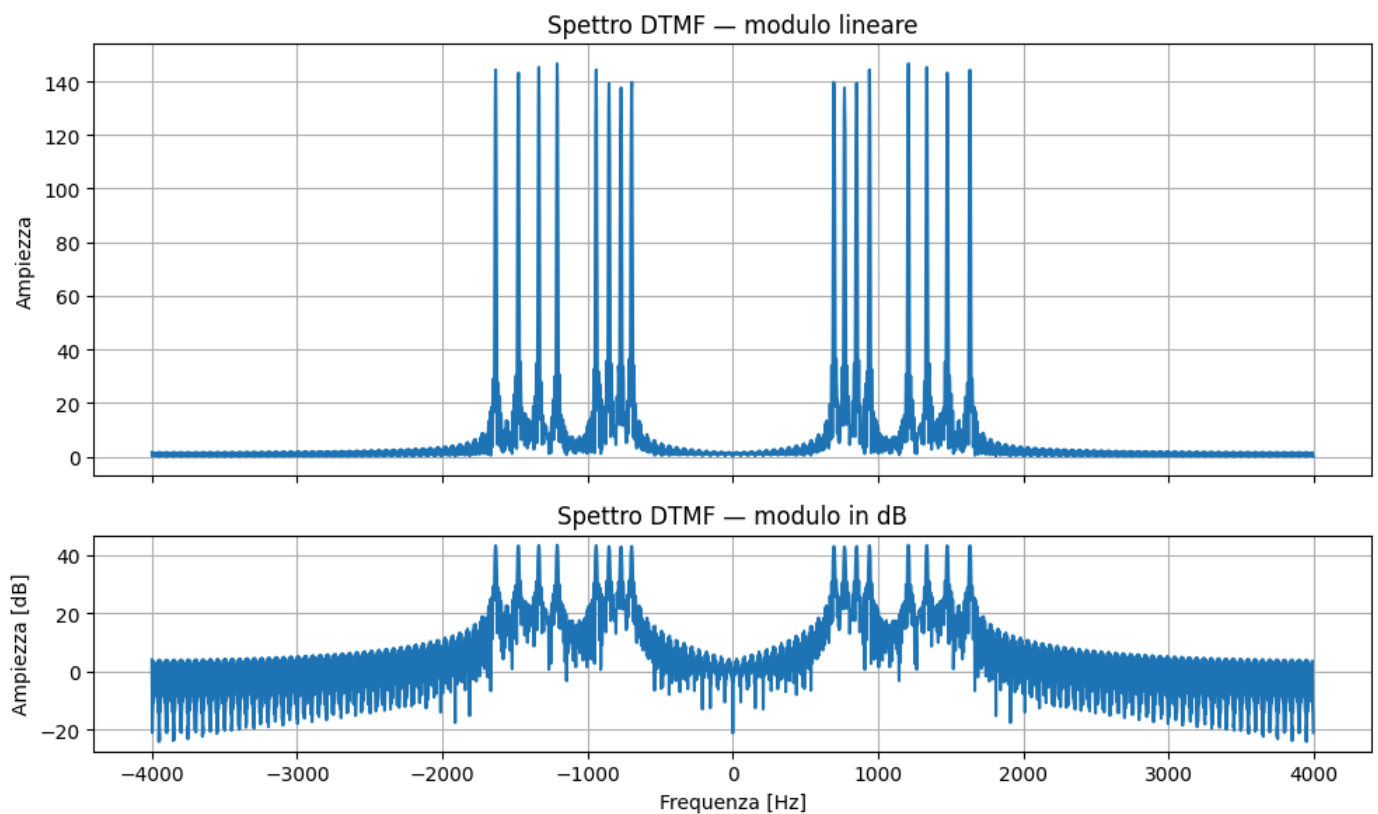
# Identifica le frequenze DTMF nel segnale
result = identify_dtmf_frequencies(x, fs_out, F1, F2, zero_pad=1, delta_hz=8.0, plot=True)
plot_signal_and_spectrum(x, fs_out, zero_pad=1)

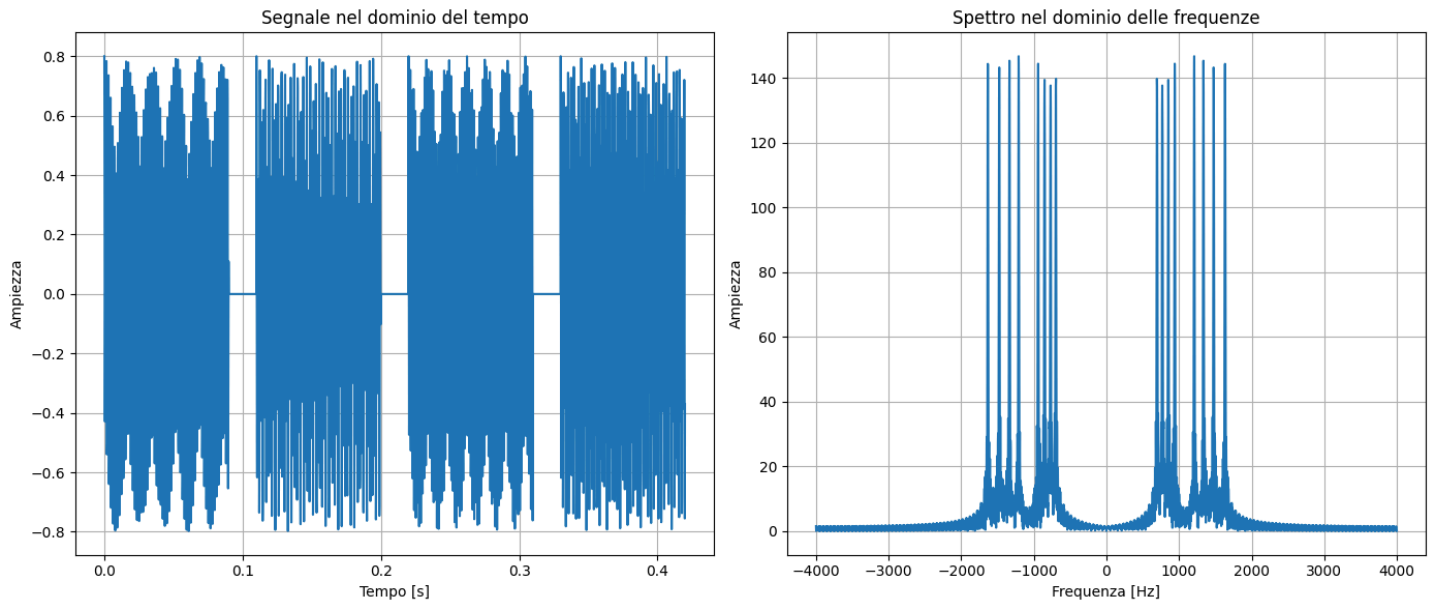
# Stampa i risultati
print("Frequenze identificate (F1):", result['F1'])
print("Frequenze identificate (F2):", result['F2'])

```



File WAV salvato in: ./analisi_spettrale\dtmf_sequence_24Chash_90ms_fs8000.wav





Frequenze identificate (F1): {697: (np.float64(695.2380952380954), np.float64(139.68377863048693)), 770: (np.float64(769.0476190476193), np.float64(137.5899835339705)), 852: (np.float64(852.3809523809523), np.float64(139.35189213093392)), 941: (np.float64(940.4761904761908), np.float64(144.32012456979538))}

Frequenze identificate (F2): {1209: (np.float64(1209.5238095238092), np.float64(146.6323315976143)), 1336: (np.float64(1335.7142857142853), np.float64(145.2363958973183)), 1477: (np.float64(1476.1904761904761), np.float64(143.16159541669165)), 1633: (np.float64(1633.3333333333333), np.float64(144.26066946988516))}

Lo **spettro** di un segnale è la sua rappresentazione nel dominio delle frequenze, ottenuta tramite la **DFT**. In pratica, l'operazione di DFT scompone il segnale nel dominio del tempo in diverse **componenti frequenziali**. Ogni componente frequenziale ha un'ampiezza (che ci dice quanto quella frequenza è presente nel segnale) e una fase (che descrive il suo sfasamento).

- Modulo lineare** Il **modulo lineare** dello spettro è semplicemente l'ampiezza delle componenti frequenziali, calcolata come il valore assoluto dei coefficienti della DFT. Viene tracciato su un grafico, dove sull'asse delle **frequenze** (sul lato x) mettiamo le frequenze che vanno da $-Fs/2$ a $Fs/2$, e sull'asse delle **ampiezze** (sul lato y) mettiamo l'ampiezza delle componenti (senza applicare alcuna trasformazione logaritmica). Questo dà una rappresentazione delle **intensità delle frequenze** nel segnale audio.
- Modulo in dB** Il **modulo in dB** significa rappresentare l'ampiezza dello spettro usando una scala logaritmica in decibel (dB). Questa scala è molto utile per visualizzare segnali che spaziano su diverse ampiezze, perché permette di vedere meglio le **frequenze deboli** che potrebbero non essere visibili su una scala lineare. I valori in decibel sono particolarmente usati in acustica, perché riflettono meglio la percezione umana delle variazioni di intensità. La formula utilizzata nel codice è:

$$X_{dB} = 20 \cdot \log_{10}(|X| + 1e^{-12})$$

Commento sui Plot

Spettro DTMF - Modulo Lineare e in dB Nel **primo** grafico, "Spettro DTMF — modulo lineare e in dB", vengono visualizzati i moduli in **scala lineare** (parte superiore) e **in dB** (parte inferiore) dello spettro del segnale DTMF. Il grafico in scala lineare mostra le ampiezze delle componenti spettrali su un'ampia gamma di frequenze, con picchi ben visibili intorno alle frequenze caratterizzanti dei toni DTMF, che sono generate da combinazioni di frequenze basse (F1) e alte (F2). La versione in **dB** consente di vedere meglio i toni più deboli che, nella versione lineare, risulterebbero poco visibili. La visualizzazione evidenzia i **picchi prominenti** che corrispondono alle frequenze delle bande DTMF, principalmente visibili attorno ai **700-1000 Hz** per la banda bassa (F1) e **1200-1700 Hz** per la banda alta (F2). La distinzione tra **modulo lineare** e **dB** permette di avere una comprensione completa delle ampiezze relative delle componenti spettrali, sia in termini assoluti che su una scala logaritmica.

Plot segnale nel dominio del tempo e della frequenza

- Segnale nel dominio del tempo (a sinistra)** Il grafico a sinistra mostra il **segno nel dominio del tempo**. Il segnale è una sequenza di **impulsi regolari**, che corrispondono ai toni DTMF generati per la sequenza "24C#". Ogni impulso è il risultato di una **combinazione di frequenze** (una bassa e una alta) che vengono emesse per ogni tasto premuto. L'ampiezza del segnale oscilla tra valori positivi e negativi, indicando che si tratta di un segnale **alternato** (tipico di segnali audio). Questo tipo di segnale è il risultato della **somma di due frequenze pure** (una per la parte bassa e una per la parte alta), che definiscono il tono.
 - Durata di ogni tono:** Ogni tono ha una durata di circa **0.09 secondi**, come definito nel parametro `toneDuration`. L'impulso visibile è quindi un'oscillazione breve, seguita da una pausa prima dell'inizio del prossimo tono.
 - Intervallo tra i toni:** C'è un **gap di 0.02 secondi** tra i toni, che è visibile come la piccola separazione tra gli impulsi.
- Spettro nel dominio delle frequenze (a destra)** Il grafico a destra mostra lo **spettro del segnale nel dominio delle frequenze**, ottenuto tramite la trasformata di Fourier (DFT). In questo grafico possiamo vedere le **componenti spettrali** del segnale. L'intensità del segnale nelle diverse frequenze è visibile sia nel **modulo lineare** che, implicitamente, nella sua distribuzione. I picchi principali sono notevolmente alti rispetto alle altre frequenze, mostrando che i toni DTMF dominano il segnale.
 - Picchi prominenti:** I picchi evidenti sono associati alle **frequenze DTMF**. In particolare, possiamo osservare picchi a **697 Hz, 770 Hz, 1209 Hz, 1336 Hz**, che corrispondono alle frequenze tipiche dei tasti DTMF (una frequenza bassa per ogni tasto, e una frequenza alta).
 - Intervallo delle frequenze:** L'asse delle frequenze va da **-4000 Hz** a **4000 Hz**, mostrando che l'analisi spettrale è stata effettuata su un intervallo sufficientemente ampio da includere sia le frequenze basse che quelle alte.

Il grafico nel dominio del **tempo** mostra chiaramente la sequenza di toni DTMF e le rispettive pause. Il grafico nel dominio della **frequenza** evidenzia chiaramente le **frequenze dominanti** (697 Hz, 770 Hz, 1209 Hz, 1336 Hz), che corrispondono ai toni emessi dai tasti DTMF, rendendo visibile la struttura del segnale audio. Questi picchi sono associati alle frequenze principali dei tasti "2", "4", "C" e "#".

In generale, questi grafici mostrano chiaramente la presenza dei toni DTMF, consentendo di identificare le frequenze corrispondenti ai tasti DTMF premuti.

È possibile identificare le frequenze dei toni?

L'analisi spettrale del segnale audio consente di identificare chiaramente le frequenze dei toni DTMF tramite la trasformata di Fourier. Nel dominio del tempo, il segnale appare come una sequenza di impulsi regolari, ciascuno associato a un tono DTMF. La durata di ciascun tono e le separazioni tra di essi sono ben visibili, mostrando chiaramente la sequenza dei tasti premuti. Tuttavia, è nel dominio delle frequenze che l'identificazione delle frequenze specifiche diventa particolarmente evidente. I picchi spettrometrici in corrispondenza delle frequenze dominanti, come **697 Hz, 770 Hz, 1209 Hz e 1336 Hz**, corrispondono esattamente alle frequenze basse e alte caratteristiche dei tasti DTMF. Questi picchi sono chiaramente separati dalle altre frequenze e hanno ampiezze significative, rendendo facile l'individuazione delle frequenze precise associate ai toni. La possibilità di osservare tali picchi distinti nel dominio delle frequenze conferma che le frequenze dei toni possono essere facilmente identificate e correlate ai tasti DTMF premuti.

Esercizio 5 – Estrazione delle frequenze

Scrivere una funzione che ricostruisca la sequenza a partire dal segnale sonoro.

```
In [10]: # Esercizio 5 - Estrazione / decodifica della sequenza DTMF
# -----
# - Ricostruzione del segnale

def reverse_tones_dict(tones: Dict[str, Tuple[int,int]] -> Dict[Tuple[int,int], str]:
    return {v: k for k, v in tones.items()}

# Segmentation utilities
def energy_based_segments(x: np.ndarray,
                          fs: int,
                          frame_ms: float = 10.0,
                          energy_threshold: float = 0.2,
                          min_tone_duration: float = 0.05) -> List[Tuple[int,int]]:
    """
    Segmenta il segnale x in intervalli attivi basandosi sull'energia.
    Restituisce una lista di tuple (start_idx, end_idx) in campioni.

    Parametri:
    - frame_ms: lunghezza frame per calcolare energia (ms)
    - energy_threshold: soglia relativa rispetto al massimo dell'energia (0..1)
    - min_tone_duration: durata minima riconosciuta come tono (s)
    """
    x = np.asarray(x).astype(float).flatten()
    N = len(x)
    if N == 0:
        return []

    frame_len = max(1, int(round(frame_ms * 1e-3 * fs)))
    # energia per frame (RMS)
    n_frames = int(np.ceil(N / frame_len))
    energy = np.zeros(n_frames)
    for k in range(n_frames):
        s = k * frame_len
        e = x[s:s + frame_len]
        energy[k] = np.sqrt(np.mean(e**2)) if e.size > 0 else 0.0

    # soglia relativa
    thr = energy_threshold * (np.max(energy) if energy.size > 0 else 1.0)

    # attività per frame (boolean array)
    active = energy >= thr

    # trova intervalli attivi contigui
    segments = []
    k = 0
    while k < n_frames:
        if active[k]:
            k_start = k
            while k < n_frames and active[k]:
                k += 1
            k_end = k # exclusive
            start_idx = k_start * frame_len
            end_idx = min(N, k_end * frame_len)
            # scarta se troppo breve
            if (end_idx - start_idx) / fs >= min_tone_duration:
                segments.append((start_idx, end_idx))
        else:
            k += 1

    return segments

def fixed_duration_segments(x: np.ndarray, fs: int,
                            toneDuration: float,
                            interToneGap: float = 0.0) -> List[Tuple[int,int]]:
    """
    Segmentazione quando la durata di ogni tono e' nota.
    Restituisce lista di (start_idx, end_idx).
    """
    if toneDuration <= 0:
        raise ValueError("toneDuration deve essere > 0 per questa segmentazione.")
    x = np.asarray(x).flatten()
    N = len(x)
```

```

tone_samples = int(round((toneDuration * fs))
gap_samples = int(round((interToneGap * fs)) if interToneGap > 0 else 0

segments = []
idx = 0
while idx < N:
    start = idx
    end = min(N, idx + tone_samples)
    segments.append((start, end))
    idx = end + gap_samples
return segments

def compute_segment_spectrum(x_seg: np.ndarray,
                             fs: int,
                             zero_pad_factor: int = 1
                             ) -> Tuple[np.ndarray, np.ndarray]:
    """
    Calcola FFT (magnitudo) del segmento x_seg, con zero-padding.
    Restituisce (freqs_shifted, mag_shifted) con axis centrato (-fs/2..fs/2).
    """
    x = np.asarray(x_seg).astype(float).flatten()
    N = len(x)
    if N == 0:
        return np.array([]), np.array([])

    w = np.ones(N) # finiestra rettangolare
    xw = x * w

    Npad = max(1, int(N * max(1, zero_pad_factor)))

    if Npad < N:
        Npad = N

    X = np.fft.fft(xw, n=Npad)
    Xs = np.fft.fftshift(X)
    Xs_mag = np.abs(Xs)

    freqs = np.fft.fftshift(np.fft.fftfreq(Npad, d=1.0/fs)) # da -fs/2 a fs/2

    return freqs, Xs_mag

def pick_best_candidate(freqs: np.ndarray,
                        mag: np.ndarray,
                        candidates: List[float],
                        delta_hz: float = 10.0) -> Dict[float, Tuple[float, float]]:
    """
    Per ogni frequenza candidata (candidates) cerca il massimo dello spettro
    nell'intervallo [cand - delta_hz, cand + delta_hz].
    Restituisce dict: cand -> (f_best, mag_best)
    """
    out = {}
    for cand in candidates:
        # trova indici validi
        idxs = np.where((freqs >= cand - delta_hz) & (freqs <= cand + delta_hz))[0]
        if idxs.size == 0:
            out[cand] = (np.nan, 0.0)
            continue
        sub = mag[idxs]
        k = np.argmax(sub)
        best_idx = idxs[k]
        out[cand] = (freqs[best_idx], mag[best_idx])
    return out

# Main decoding function
def decode_dtmf(x: np.ndarray,
                fs: int,
                toneDuration: Optional[float] = None,
                interToneGap: float = 0.05,
                tones: Optional[Dict[str, Tuple[int,int]]] = None,
                F1: Optional[List[float]] = None,
                F2: Optional[List[float]] = None,
                segmentation: str = 'auto',
                energy_frame_ms: float = 10.0,
                energy_threshold: float = 0.2,
                zero_pad_factor: int = 1,
                delta_hz: float = 10.0,
                plot_segments: bool = False) -> Tuple[str, List[Tuple[str, float]]]:
    """
    Decodifica la sequenza DTMF dal segnale mono x.

    Parametri principali:
    - x: segnale mono (1D numpy array)
    - fs: sample rate
    - toneDuration: se noto, durata di ciascun tono (s). Se fornito e segmentation='fixed' verrà usato
    - interToneGap: pausa stimata tra i toni (s) se segmentation='fixed'
    - tones, F1, F2: mappature DTMF (se None verranno costruite internamente)
    - segmentation: 'auto' (energia) o 'fixed' (toneDuration obbligatorio)
    - energy_frame_ms, energy_threshold: parametri per segmentazione per energia
    - zero_pad_factor: fattore di zero-padding per migliorare risoluzione FFT
    - delta_hz: tolleranza per la ricerca picco intorno a ciascuna frequenza candidata
    - plot_segments: se True plotti lo spettro per ogni segmento (debug)

```

```

Restituisce:
- decoded: stringa dei tasti decodificati
- details: lista di tuple (tasto, conf) dove conf e' ampiezza normalizzata (per confronto)
"""

# mappature di default
if tones is None:
    tones = build_tones_dict()
if F1 is None or F2 is None:
    F1, F2 = build_frequency_arrays()
inv_tones = reverse_tones_dict(tones)

x = np.asarray(x).flatten()
N = len(x)
if N == 0:
    return "", []

# Segmentazione
if segmentation == 'fixed':
    if toneDuration is None:
        raise ValueError("Per segmentation='fixed' toneDuration deve essere fornito.")
    segments = fixed_duration_segments(x, fs, toneDuration, interToneGap)
else:
    # auto energy based
    segments = energy_based_segments(x, fs, frame_ms=energy_frame_ms,
                                     energy_threshold=energy_threshold,
                                     min_tone_duration=(toneDuration if toneDuration is not None else 0.05))

decoded_chars = []
details: List[Tuple[str, float]] = []

# per ciascun segmento analizza spettro e trova f_low e f_high
for seg_idx, (samp_start, samp_end) in enumerate(segments):
    seg = x[samp_start:samp_end]
    if seg.size == 0:
        continue

    freqs, mag = compute_segment_spectrum(seg, fs, zero_pad_factor=zero_pad_factor)

    # scegli migliori candidati attorno a F1 e F2
    meas1 = pick_best_candidate(freqs, mag, F1, delta_hz=delta_hz)
    meas2 = pick_best_candidate(freqs, mag, F2, delta_hz=delta_hz)

    # trova la f_low con magnitudine massima tra candidati F1
    best_f1, best_mag1, best_cand1 = np.nan, 0.0, None
    for cand, (f_found, m_found) in meas1.items():
        if m_found > best_mag1:
            best_mag1 = m_found
            best_f1 = f_found
            best_cand1 = cand

    # trova la f_high con magnitudine massima tra candidati F2
    best_f2, best_mag2, best_cand2 = np.nan, 0.0, None
    for cand, (f_found, m_found) in meas2.items():
        if m_found > best_mag2:
            best_mag2 = m_found
            best_f2 = f_found
            best_cand2 = cand

    # normalizza conf (rapporto tra l'ampiezza trovata e la somma delle due)
    total_mag = best_mag1 + best_mag2 if (best_mag1 + best_mag2) > 0 else 1.0
    conf = (best_mag1 + best_mag2) / total_mag

    # map frequenze trovate alle coppie di indici più vicine (prendiamo canditore scelto)
    if best_cand1 is None or best_cand2 is None:
        decoded = '?'
    else:
        try:
            i = F1.index(int(best_cand1))
            j = F2.index(int(best_cand2))
            decoded = inv_tones.get((i, j), '?')
        except ValueError:
            # fallback: cerca l'indice più vicino
            i = int(np.argmin([abs(best_f1 - f) for f in F1]))
            j = int(np.argmin([abs(best_f2 - f) for f in F2]))
            decoded = inv_tones.get((i, j), '?')

    decoded_chars.append(decoded)
    # confidence metric: rapporto tra (best_mag1+best_mag2) e total_mag
    conf_metric = (best_mag1 + best_mag2) / (total_mag + 1e-12)
    details.append((decoded, float(conf_metric)))

# plotting per segmento (debug)
if plot_segments:
    plt.figure(figsize=(8, 3))
    plt.plot(freqs, mag)
    plt.title(f"Segment {seg_idx}: {decoded} | best low {best_cand1}Hz @ {best_f1:.1f}Hz (mag={best_mag1:.1f}), "
              f"best high {best_cand2}Hz @ {best_f2:.1f}Hz (mag={best_mag2:.1f})")
    plt.xlabel("Freq [Hz]")
    plt.ylabel("Mag")
    plt.grid(True)
    plt.xlim(min(F1)-100, max(F2)+200)
    plt.show()

```

```
decoded_str = "".join(decoded_chars)
return decoded_str, details
```

```
In [11]: # ESEMPIO
decoded, details = decode_dtmf(x, fs, toneDuration=toneDuration, segmentation='fixed',
                               interToneGap=interToneGap, zero_pad_factor=1,
                               delta_hz=8.0, plot_segments=False)
print("Decodificato:", decoded)
print("Dettagli:", details)
```

Decodificato: 24C#

Dettagli: [('2', 0.999999999999996), ('4', 0.9999999999999961), ('C', 0.9999999999999961), ('#', 0.9999999999999961)]

La variabile **conf** è una misura di confidenza che indica l'affidabilità della decodifica di un tono DTMF. Viene calcolata come il rapporto tra la somma delle ampiezze delle due frequenze identificate nel segmento di segnale e la somma totale delle ampiezze nel segmento. La formula per **conf** è:

$$\text{conf} = \frac{\text{best_mag1} + \text{best_mag2}}{\text{total_mag}}$$

dove:

- **best_mag1** è l'ampiezza della frequenza bassa (F1),
- **best_mag2** è l'ampiezza della frequenza alta (F2),
- **total_mag** è la somma delle ampiezze di tutte le frequenze nel segmento.

Il valore di **conf** è normalizzato e compreso tra **0** e **1**. Un valore vicino a **1** indica una rilevazione chiara e affidabile del tono, mentre un valore vicino a **0** indica una decodifica incerta o imprecisa. La confidenza aiuta a valutare quanto le frequenze identificate siano veramente rappresentative del tono DTMF, fornendo una misura utile per distinguere tra tasti decodificati con alta precisione e quelli meno affidabili.

Esercizio 6 – Valutazione degli effetti sui parametri

Valutare l'effetto delle seguenti variabili sulla **qualità della generazione** e del **riconoscimento** della sequenza. In particolare, eseguire esperimenti per analizzare l'effetto di:

- **Variazione della durata del tono** Generare diversi toni per durate differenti (es. `toneDuration` variabile). Per ciascun caso:
 - rappresentare il segnale nel dominio del tempo;
 - calcolare lo spettro di ampiezza (modulo della DFT);
 - commentare i risultati osservati (risoluzione in frequenza, leakage, possibilità di identificare le componenti DTMF).
- **Zero-padding** Applicare la DFT con e senza zero-padding per diversi livelli di riempimento (es. fattori 1, 2, 4, 8) per confrontare la risoluzione in frequenza. Commentare come cambia la precisione nell'identificazione dei picchi spettrali e gli effetti sullo smoothing/spreading del segnale.
- **Aggiunta di rumore al segnale** Aggiungere rumore gaussiano con diversi valori di deviazione standard per osservare come cambia la qualità del segnale e l'accuratezza del riconoscimento. Suggerimenti per l'analisi:
 - Grafico 1: confrontare in dominio temporale il segnale originale e il segnale con rumore aggiunto (per visualizzare le differenze);
 - Grafico 2: mostrare l'effetto di diversi livelli di rumore (basso, medio, alto) sul segnale nel tempo;
 - Grafico 3: confrontare la DFT del segnale originale e del segnale rumoroso per evidenziare l'aumento dell'ampiezza in frequenze non desiderate;
 - Grafico 4: confrontare l'accuratezza del riconoscimento (espressa in percentuale) con e senza rumore, per vari livelli di deviazione standard;
 - Facoltativo: applicare tecniche di denoising (filtri, median/ spectral subtraction, ecc.) e confrontare l'accuratezza del riconoscimento prima/dopo il denoising.
- **Variazione della frequenza di campionamento** Campionare il segnale DTMF a diverse frequenze (`Fs`) e confrontare il segnale originale con i segnali campionati. Per ciascun caso:
 - confrontare l'accuratezza del riconoscimento ai vari livelli di campionamento (es. 8000 Hz, 16000 Hz, 44100 Hz);
 - evidenziare la degradazione dovuta al sottocampionamento (aliasing) e i miglioramenti con oversampling;
 - studiare e proporre tecniche per evitare aliasing (filtraggio anti-aliasing prima del downsampling) e tecniche di filtraggio per migliorare il riconoscimento.

6- Variazione durata tono

```
In [12]: # Esercizio 6 - Valutazione degli effetti sui parametri
# -----
# - Variazione durata tono
# - Zero-padding
# - Introduzione del rumore
# - Variazione della frequenza di campionamento

# 1- Variazione durata tono -> t=0.01s, t=0.1s, t=1s
# Parametri
numbers = "24C#"
toneDuration = 0.09 # secondi per tono
fs = 8000
interToneGap = 0.02 # 50 ms tra toni
out_dir = "./esperimenti/variazione_tono" # cartella di salvataggio

durations = [0.01, 0.1, 0.5] # Durate dei toni in secondi

# Generazione dei segnali DTMF per le diverse durate
plt.figure(figsize=(12, 8))

# Loop per ciascuna durata
for i, duration in enumerate(durations):
```

```

# a- Generazione segnale 24C#
x, fs_out, wav_path = dialNumber(numbers, duration, fs=fs,
                                interToneGap=interToneGap,
                                amplitude=0.8,
                                out_dir=out_dir,
                                plot = False)

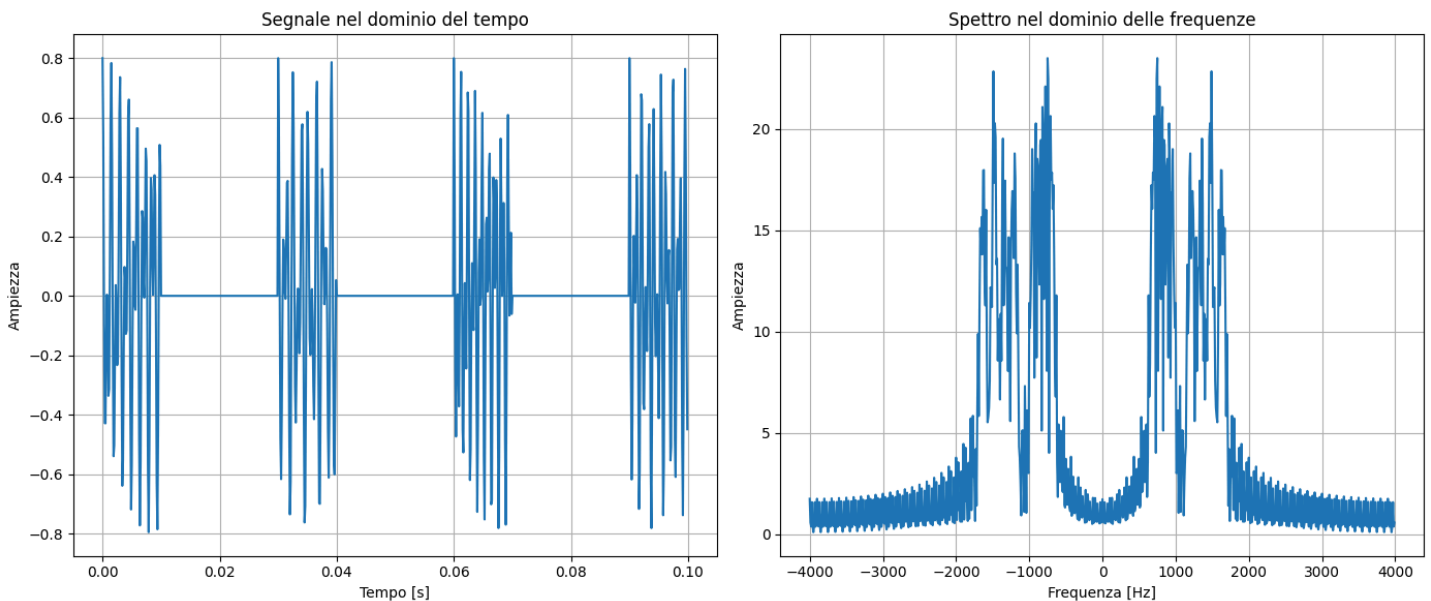
# b- Calcolo dello spettro DFT
# Frequenze basse (F1) e alte (F2) per DTMF
F1, F2 = build_frequency_arrays()
# Identifica le frequenze DTMF nel segnale
result = identify_dtmf_frequencies(x, fs_out, F1, F2, zero_pad=1, delta_hz=8.0, plot=False)

# c- Plot segnale nel dominio del tempo e nel dominio della frequenza
plot_signal_and_spectrum(x, fs_out, zero_pad=1)
# Stampa i risultati
print("Frequenze identificate (F1):", result['F1'])
print("Frequenze identificate (F2):", result['F2'])

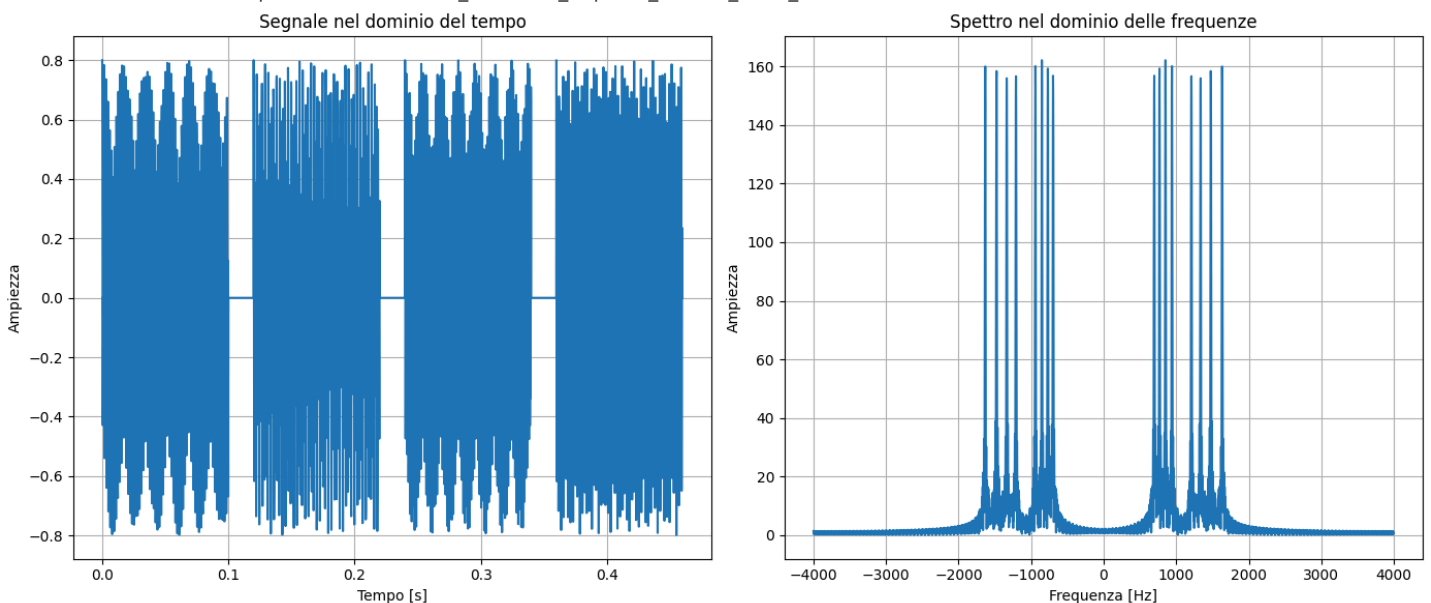
# d- Decodifica del tono
decoded, details = decode_dtmf(x, fs, toneDuration=duration, segmentation='fixed',
                              interToneGap=interToneGap, zero_pad_factor=1,
                              delta_hz=8.0, plot_segments=False)
print("Decodificato:", decoded)
print("Dettagli:", details)

```

File WAV salvato in: ./esperimenti/variazione_tono\dtmf_sequence_24Chash_10ms_fs8000.wav
 <Figure size 1200x800 with 0 Axes>



Frequenze identificate (F1): {697: (np.float64(690.0), np.float64(17.844957672869707)), 770: (np.float64(770.0), np.float64(16.87641198233032)), 852: (np.float64(850.0), np.float64(19.210832739010563)), 941: (np.float64(940.0), np.float64(16.840190519660826))}
 Frequenze identificate (F2): {1209: (np.float64(1210.0), np.float64(13.622207998199839)), 1336: (np.float64(1330.0), np.float64(17.447592036437598)), 1477: (np.float64(1470.0), np.float64(20.280959272896606)), 1633: (np.float64(1630.0), np.float64(17.950739586373714))}
 Decodificato: ???
 Dettagli: [('?', 0.9999999999999399), ('?', 0.9999999999998183), ('?', 0.9999999999997065), ('?', 0.9999999999995127)]
 File WAV salvato in: ./esperimenti/variazione_tono\dtmf_sequence_24Chash_10ms_fs8000.wav



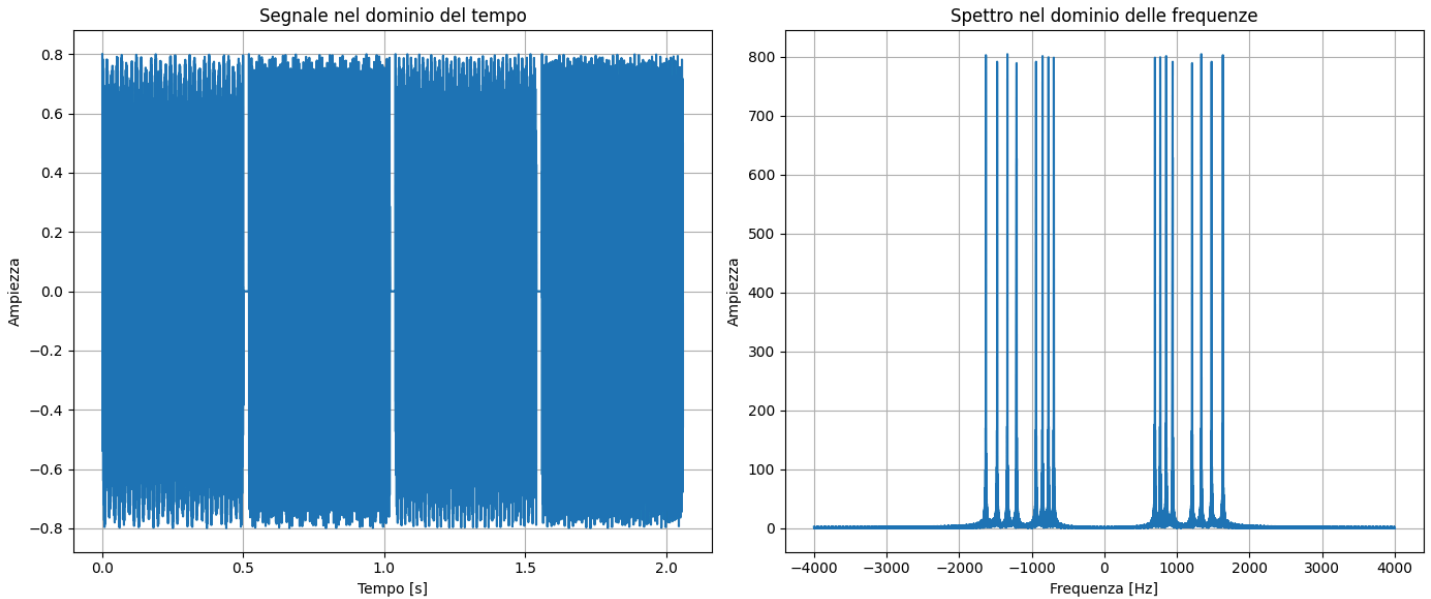
Frequenze identificate (F1): {697: (np.float64(697.8260869565211), np.float64(156.80207832626473)), 770: (np.float64(769.565217391304), np.float64(159.23573887483084)), 852: (np.float64(852.173913043478), np.float64(162.08632637969276)), 941: (np.float64(941.304347826087), np.float64(160.05466101856703))}

Frequenze identificate (F2): {1209: (np.float64(1208.695652173913), np.float64(156.5868507782125)), 1336: (np.float64(1336.95652173913), np.float64(155.91703186916192)), 1477: (np.float64(1476.086956521739), np.float64(158.36763255890295)), 1633: (np.float64(1632.608695652174), np.float64(159.94283316833477))}

Decodificato: 24C#

Dettagli: [('2', 0.999999999999996), ('4', 0.999999999999968), ('C', 0.999999999999964), ('#', 0.999999999999966)]

File WAV salvato in: ./esperimenti/variazione_tono\dtmf_sequence_24Chash_500ms_fs8000.wav



Frequenze identificate (F1): {697: (np.float64(697.0873786407765), np.float64(798.3757137011487)), 770: (np.float64(769.9029126213591), np.float64(799.0885519593988)), 852: (np.float64(851.941747572816), np.float64(801.0823046639606)), 941: (np.float64(940.7766990291266), np.float64(791.4116946224418))}

Frequenze identificate (F2): {1209: (np.float64(1209.2233009708734), np.float64(789.0566308081928)), 1336: (np.float64(1335.922330097087), np.float64(804.3580033416599)), 1477: (np.float64(1477.1844660194174), np.float64(791.3938091592418)), 1633: (np.float64(1633.0097087378645), np.float64(802.6012830892473))}

Decodificato: 24C#

Dettagli: [('2', 0.999999999999993), ('4', 0.999999999999993), ('C', 0.999999999999993), ('#', 0.999999999999999)]

Commenti sui Plot nel Dominio del Tempo

- **Durata 0.01 s:** Il segnale appare molto frastagliato e discontinuo. Ogni tono ha una durata troppo breve, che rende difficile una definizione chiara dei toni. La frequenza sembra più sfocata nel tempo.
- **Durata 0.1 s:** Il segnale è più stabile e meno frastagliato, ma ancora non perfettamente definito. La continuità del tono è migliorata rispetto a 0.01 s, ma la risoluzione non è ottimale.
- **Durata 0.5 s:** Il segnale è chiaramente definito. La forma dell'onda sinusoidale è ben visibile, con una maggiore stabilità e continuità. La risoluzione in frequenza è migliorata significativamente.

Risoluzione in Frequenza

- La risoluzione in frequenza dipende direttamente dalla durata del tono. Con una durata più breve (0.01 s), la risoluzione in frequenza diminuisce, portando a una maggiore dispersione spettrale e "leakage". Con toni più lunghi (0.1 s e 0.5 s), la risoluzione aumenta, e le frequenze sono più facilmente identificabili.

Leakage

- Il "leakage" è più evidente con la durata di 0.01 s, dove la finestra temporale non è abbastanza lunga da catturare la tonalità in modo preciso. Le frequenze si "spargono" in tutta la banda spettrale, confondendo i toni.
- Con la durata di 0.1 s, il leakage è visibile ma meno significativo. La finestra temporale è sufficiente per evitare una dispersione eccessiva delle frequenze.
- Con la durata di 0.5 s, il leakage è minimo, e ogni tono è chiaramente definito senza sfocature.

Identificazione del Segnale

- **Durata 0.01 s:** La decodifica dei toni è imprecisa e, in alcuni casi, impossibile. Le frequenze non sono ben definite.
- **Durata 0.1 s:** L'identificazione migliora, ma non è ancora perfetta. Alcuni toni sono difficili da identificare a causa della risoluzione insufficiente.
- **Durata 0.5 s:** La decodifica è molto precisa. I toni sono facilmente identificabili e associabili ai tasti DTMF corretti. La maggiore durata permette una discriminazione netta delle frequenze.

A partire da una durata di 0.1 s, il segnale DTMF risulta sufficientemente chiaro per l'identificazione, ma i migliori risultati si ottengono con durate più lunghe (0.5 s). La risoluzione in frequenza migliora notevolmente, riducendo il leakage e permettendo un riconoscimento più accurato dei toni.

6- Zero padding

```
In [18]: # 2- Zero padding -> confronto segnale con e senza padding
# Parametri
numbers = "24C#"
toneDuration = 0.05 # secondi per tono
fs = 8000
interToneGap = 0.01 # 50 ms tra toni
zero_pad_values = [1, 2, 4, 8]

# Generazione dei segnali DTMF per le diverse durate
plt.figure(figsize=(12, 8))
```

```

# Loop per ciascuna durata
for i, padding in enumerate(zero_pad_values):
    out_dir = f"./esperimenti/zero_padding/{padding}" # cartella di salvataggio
    print(f"\n--- Esperimento con zero padding factor = {padding} ---")
    # a- Generazione segnale 24C#
    x, fs_out, wav_path = dialNumber(numbers, toneDuration, fs=fs,
                                     interToneGap=interToneGap,
                                     amplitude=0.8,
                                     out_dir=out_dir,
                                     plot = False)

    # b- Calcolo dello spettro DFT
    # Frequenze basse (F1) e alte (F2) per DTMF
    F1, F2 = build_frequency_arrays()
    # Identifica le frequenze DTMF nel segnale
    result = identify_dtmf_frequencies(x, fs_out, F1, F2, zero_pad=padding, delta_hz=8.0, plot=False)

    # c- Plot segnale nel dominio del tempo e nel dominio della frequenza
    plot_signal_and_spectrum(x, fs_out, zero_pad=1)
    # Stampa i risultati
    print("Frequenze identificate (F1):", result['F1'])
    print("Frequenze identificate (F2):", result['F2'])

    # d- Decodifica del tono
    decoded, details = decode_dtmf(x, fs, toneDuration=toneDuration, segmentation='fixed',
                                   interToneGap=interToneGap, zero_pad_factor=padding,
                                   delta_hz=8.0, plot_segments=False)

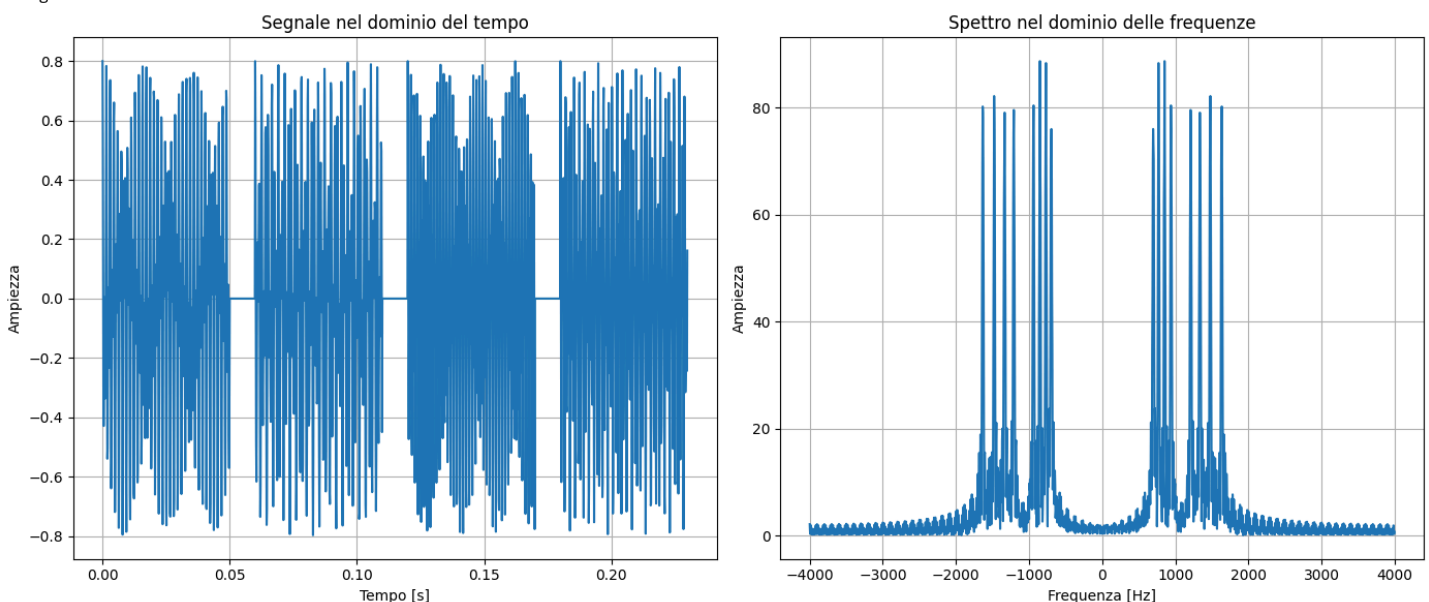
    print("Decodificato:", decoded)
    print("Dettagli:", details)

```

--- Esperimento con zero padding factor = 1 ---

File WAV salvato in: ./esperimenti/zero_padding/1\dtmf_sequence_24Chash_50ms_fs8000.wav

<Figure size 1200x800 with 0 Axes>



Frequenze identificate (F1): {697: (np.float64(695.652173913043), np.float64(76.01836259219529)), 770: (np.float64(769.565217391304), np.float64(88.35074863452027)), 852: (np.float64(852.173913043478), np.float64(88.71153465300095)), 941: (np.float64(939.1304347826081), np.float64(80.42581075928233))}

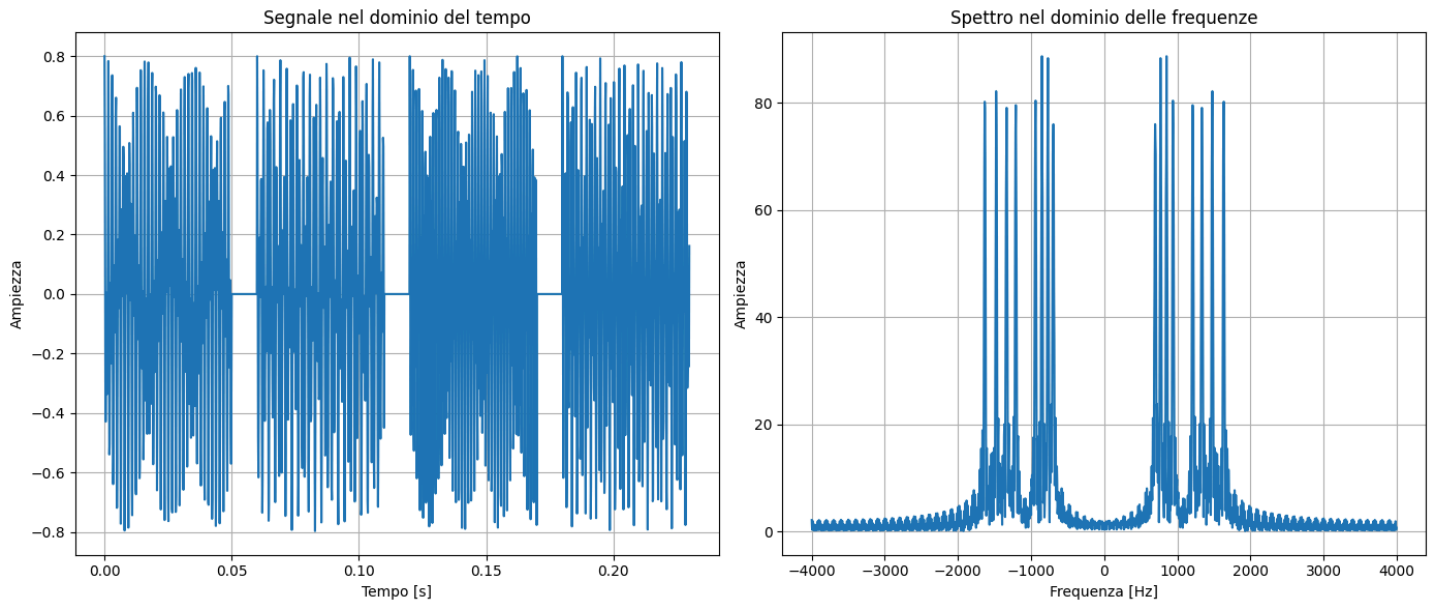
Frequenze identificate (F2): {1209: (np.float64(1208.695652173913), np.float64(79.57677332921043)), 1336: (np.float64(1334.782608695652), np.float64(79.08668483101069)), 1477: (np.float64(1478.260869565217), np.float64(82.18458524157846)), 1633: (np.float64(1634.782608695652), np.float64(80.22791898020394))}

Decodificato: 22C#

Dettagli: [('2', 0.9999999999999934), ('2', 0.9999999999999216), ('C', 0.999999999999921), ('#', 0.999999999999937)]

--- Esperimento con zero padding factor = 2 ---

File WAV salvato in: ./esperimenti/zero_padding/2\dtmf_sequence_24Chash_50ms_fs8000.wav



Frequenze identificate (F1): {697: (np.float64(702.173913043478), np.float64(78.27088978864542)), 770: (np.float64(769.565217391304), np.float64(88.35074863452027)), 852: (np.float64(852.173913043478), np.float64(88.71153465300095)), 941: (np.float64(939.1304347826081), np.float64(80.42581075928233))}

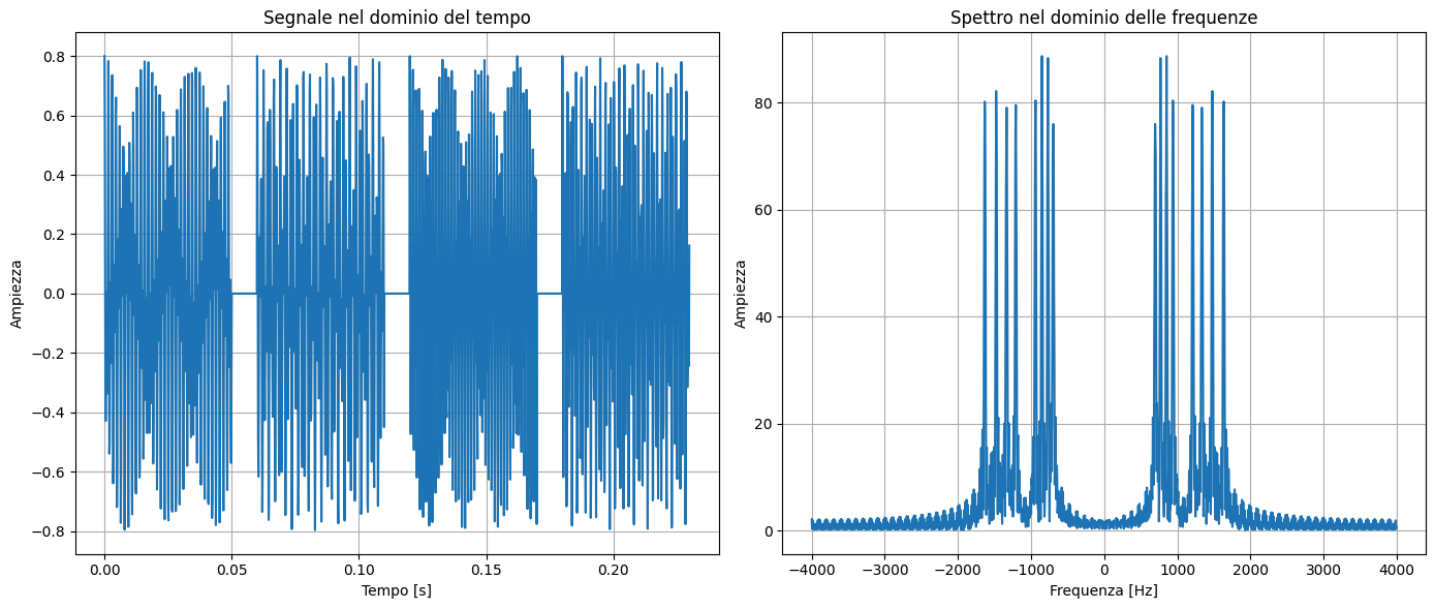
Frequenze identificate (F2): {1209: (np.float64(1208.695652173913), np.float64(79.57677332921043)), 1336: (np.float64(1332.608695652174), np.float64(80.60797970446696)), 1477: (np.float64(1478.260869565217), np.float64(82.18458524157846)), 1633: (np.float64(1634.782608695652), np.float64(80.22791898020394))}

Decodificato: 24C#

Dettagli: [('2', 0.9999999999999934), ('4', 0.9999999999999938), ('C', 0.9999999999999937), ('#', 0.9999999999999937)]

--- Esperimento con zero padding factor = 4 ---

File WAV salvato in: ./esperimenti/zero_padding/4\dtmf_sequence_24Chash_50ms_fs8000.wav



Frequenze identificate (F1): {697: (np.float64(702.173913043478), np.float64(78.27088978864543)), 770: (np.float64(769.565217391304), np.float64(88.3507486345203)), 852: (np.float64(852.173913043478), np.float64(88.71153465300094)), 941: (np.float64(939.1304347826081), np.float64(80.42581075928233))}

Frequenze identificate (F2): {1209: (np.float64(1208.695652173913), np.float64(79.57677332921043)), 1336: (np.float64(1333.695652173913), np.float64(80.92826188231791)), 1477: (np.float64(1477.173913043478), np.float64(83.11678444600798)), 1633: (np.float64(1633.695652173913), np.float64(80.38097031989949))}

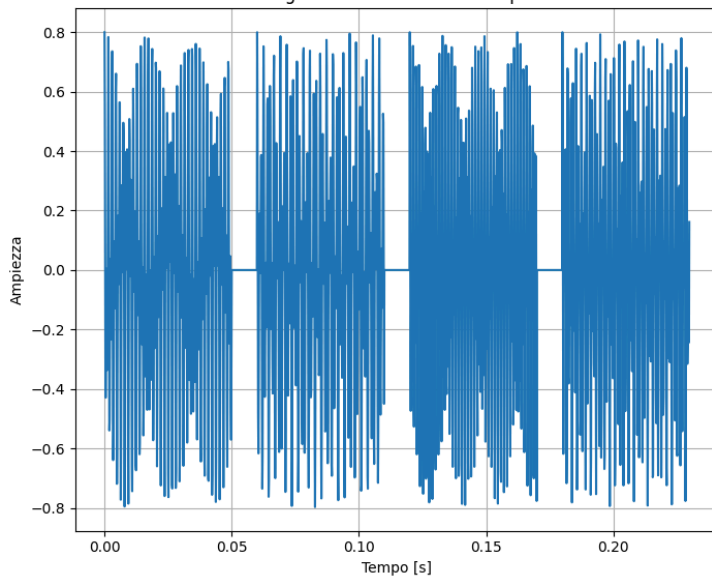
Decodificato: 24C#

Dettagli: [('2', 0.9999999999999937), ('4', 0.9999999999999938), ('C', 0.9999999999999937), ('#', 0.9999999999999937)]

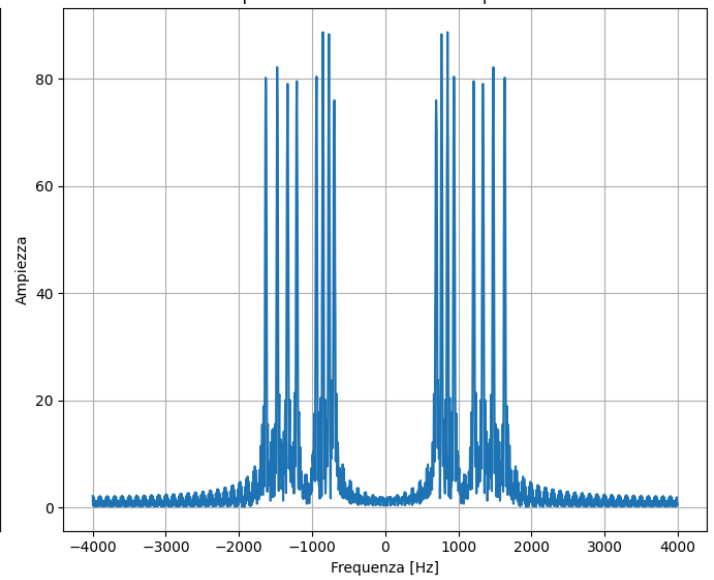
--- Esperimento con zero padding factor = 8 ---

File WAV salvato in: ./esperimenti/zero_padding/8\dtmf_sequence_24Chash_50ms_fs8000.wav

Segnale nel dominio del tempo



Spettro nel dominio delle frequenze



Frequenze identificate (F1): {697: (np.float64(702.173913043478), np.float64(78.27088978864543)), 770: (np.float64(769.021739130435), np.float64(88.70206153711058)), 852: (np.float64(852.173913043478), np.float64(88.71153465300094)), 941: (np.float64(939.673913043478), np.float64(80.51956823364216))}

Frequenze identificate (F2): {1209: (np.float64(1209.239130434782), np.float64(79.80031471207208)), 1336: (np.float64(1333.152173913043), np.float64(81.29489048874575)), 1477: (np.float64(1477.173913043478), np.float64(83.11678444600798)), 1633: (np.float64(1634.239130434782), np.float64(80.47694740917382))}

Decodificato: 24C#

Dettagli: [('2', 0.9999999999999938), ('4', 0.9999999999999938), ('C', 0.9999999999999938), ('#', 0.9999999999999938)]

MANCANO COMMENTI

6- Introduzione del rumore

In [14]:

```
# 3- Introduzione del rumore
# Funzione per creare il filtro passa basso (Butterworth)
def butter_lowpass(cutoff, fs, order=5):
    nyquist = 0.5 * fs
    normal_cutoff = cutoff / nyquist
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    return b, a

# Funzione per applicare il filtro passa basso al segnale
def lowpass_filter(data, cutoff, fs, order=5):
    b, a = butter_lowpass(cutoff, fs, order)
    return filtfilt(b, a, data)

# Funzione per la sottrazione spettrale
def spectral_subtraction(x_noisy, fs, noise_estimation_window=2048):
    """Implementazione della sottrazione spettrale per denoising."""
    # Calcola lo spettro del segnale rumoroso usando STFT
    nfft = 2048
    f, t, Zxx = stft(x_noisy, fs, nperseg=nfft, noverlap=nfft//2)

    # Stima del rumore: consideriamo la parte bassa dell'energia come rumore
    noise_spectrum = np.mean(np.abs(Zxx[:, :noise_estimation_window]), axis=1)

    # Sottrazione del rumore dallo spettro del segnale
    Zxx_clean = np.maximum(np.abs(Zxx) - noise_spectrum[:, None], 0) * np.exp(1j * np.angle(Zxx))

    # Trasformata inversa per ottenere il segnale denoised
    _, x_denoised = istft(Zxx_clean, fs, nperseg=nfft, noverlap=nfft//2)
    return x_denoised

# Funzione per plottare il segnale nel dominio del tempo e nello spettro (affiancati)
def plot_signals(t: np.ndarray, x_clean: np.ndarray, x_noisy: np.ndarray, fs: int, noise_std: float):
    """Plotta segnale e segnale + rumore affiancati (dominio del tempo e della frequenza)."""

    print(f"\nDeviato standard rumore = {noise_std}")

    # Plot 1: Tempo - Segnale e Segnale + Rumore
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(t, x_clean, label="Segnale Pulito")
    plt.plot(t, x_noisy, label=f"Segnale con Rumore (σ={noise_std})", alpha=0.6)
    plt.title(f"Domini del Tempo: Rumore σ={noise_std}")
    plt.xlabel("Tempo [s]")
    plt.ylabel("Ampiezza")
    plt.legend()
    plt.grid(True)

    # Plot 2: Frequenza - DFT del segnale e del segnale con rumore
    freqs_clean, mag_clean = compute_spectrum(x_clean, fs)
    freqs_noisy, mag_noisy = compute_spectrum(x_noisy, fs)

    plt.subplot(1, 2, 2)
    plt.plot(freqs_clean, 20 * np.log10(mag_clean + 1e-12), label="DFT Segnale Pulito")
```

```

plt.plot(freqs_noisy, 20 * np.log10(mag_noisy + 1e-12), label=f"DFT Segnale + Rumore ( $\sigma$ ={noise_std})", alpha=0.6)
plt.title(f"Domini della Frequenza: Rumore  $\sigma$ ={noise_std}")
plt.xlabel("Frequenza [Hz]")
plt.ylabel("Ampiezza [dB]")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Funzione per eseguire gli esperimenti con rumore a diversi livelli
def experiment_noise_levels(sequence: str = "24C#",
                           toneDuration: float = 0.5,
                           fs: int = 8000,
                           noise_std: List[float] = [0.01, 0.05, 0.1, 0.5],
                           trials: int = 5,
                           out_dir: str = "./esperimenti/rumore"):
    """
    Genera la sequenza, aggiunge rumore gaussiano a diversi livelli e valuta l'accuratezza.
    Ritorna risultati aggregati per livello di rumore.
    """
    os.makedirs(out_dir, exist_ok=True)
    x_clean, fs, _ = dialNumber(sequence, toneDuration, fs=fs, interToneGap=0.05, amplitude=0.8, out_dir=out_dir)
    t = np.arange(len(x_clean)) / fs

    results = []

    # Loop su vari livelli di rumore
    for sigma in noise_std:
        # Aggiunge rumore gaussiano UNA sola volta per livello di rumore
        noise = np.random.normal(0, sigma, size=x_clean.shape)
        x_noisy = x_clean + noise

        # Plot dei segnali (tempo e frequenza)
        plot_signals(t, x_clean, x_noisy, fs, sigma)

        # Decodifica del segnale rumoroso
        decoded, details = decode_dtmf(x_noisy, fs, toneDuration=toneDuration, segmentation='fixed', zero_pad_factor=1)
        print(f"Decodifica senza denoising: {decoded}")

        # Salva il segnale rumoroso
        fname_noisy = os.path.join(out_dir, f"seq_{sequence}_noise_{int(sigma*1000)}_fs{fs}_noisy.wav")
        wavfile.write(fname_noisy, fs, np.int16(np.clip(x_noisy, -1, 1)*32767))

        # Denoising: applica sottrazione spettrale per ridurre il rumore
        x_denoised = spectral_subtraction(x_noisy, fs)

        # Decodifica del segnale denoised
        decoded_denoised, details_denoised = decode_dtmf(x_denoised, fs, toneDuration=toneDuration, segmentation='fixed', zero_pad_factor=1)
        print(f"Decodifica con denoising: {decoded_denoised}")

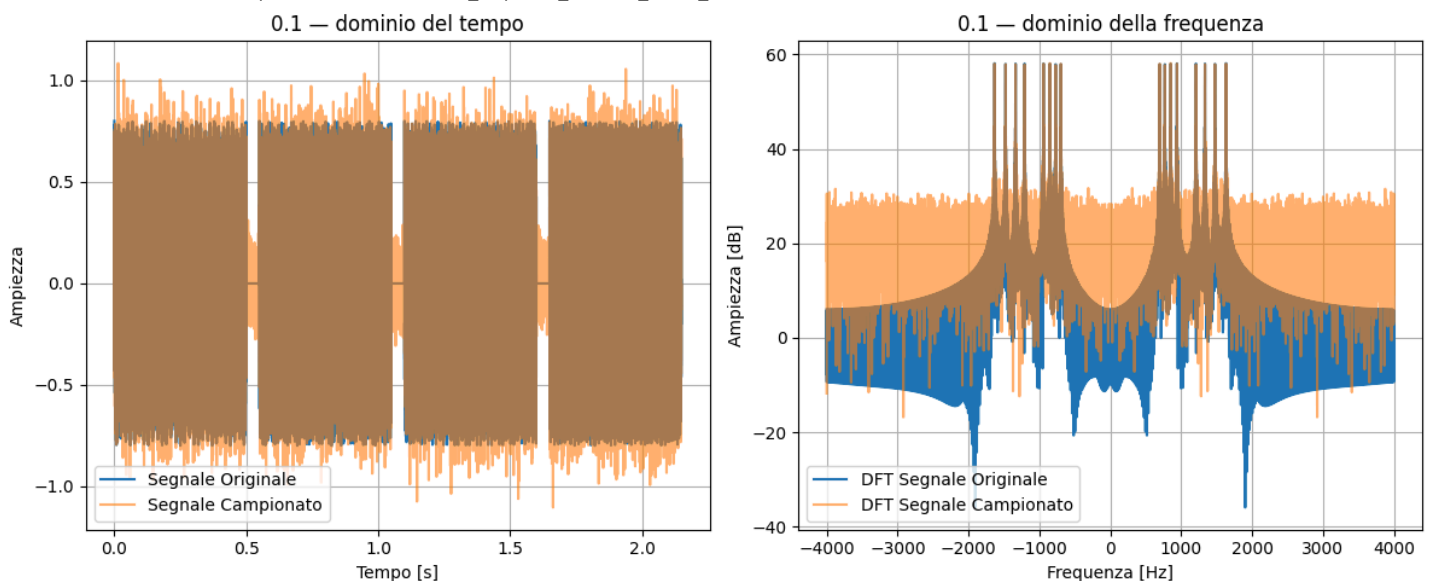
        # Salva il segnale denoised
        fname_denoised = os.path.join(out_dir, f"seq_{sequence}_denoised_noise_{int(sigma*1000)}_fs{fs}.wav")
        wavfile.write(fname_denoised, fs, np.int16(np.clip(x_denoised, -1, 1)*32767))

    return _

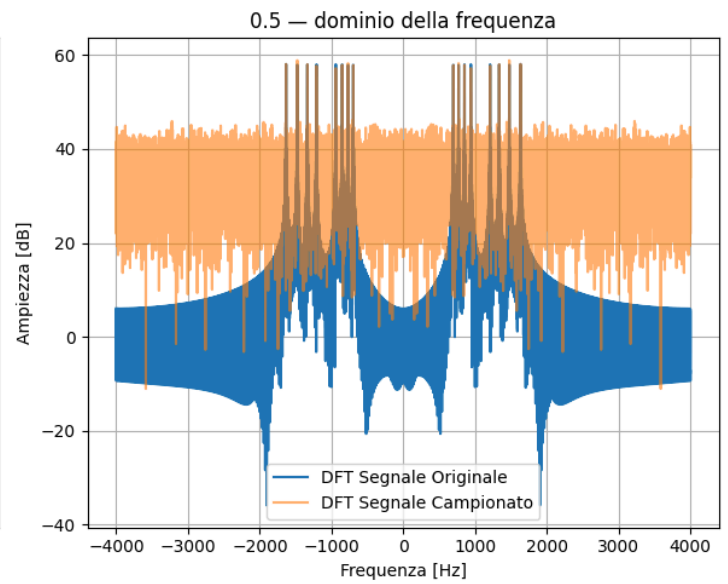
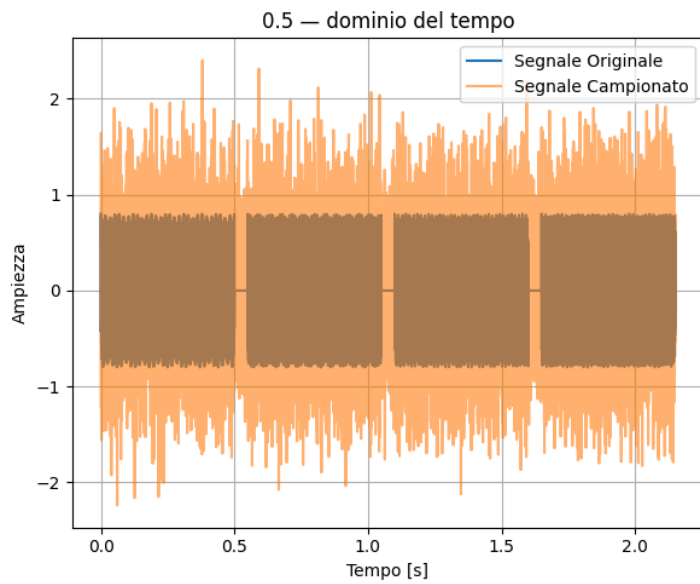
```

In [22]: # Esecuzione dell'esperimento con livelli di rumore e denoising
res_noise = experiment_noise_levels(sequence="24C#", toneDuration=0.5, fs=8000, noise_std=[0.1, 0.5, 1, 5], out_dir="./esperimenti/rumore")

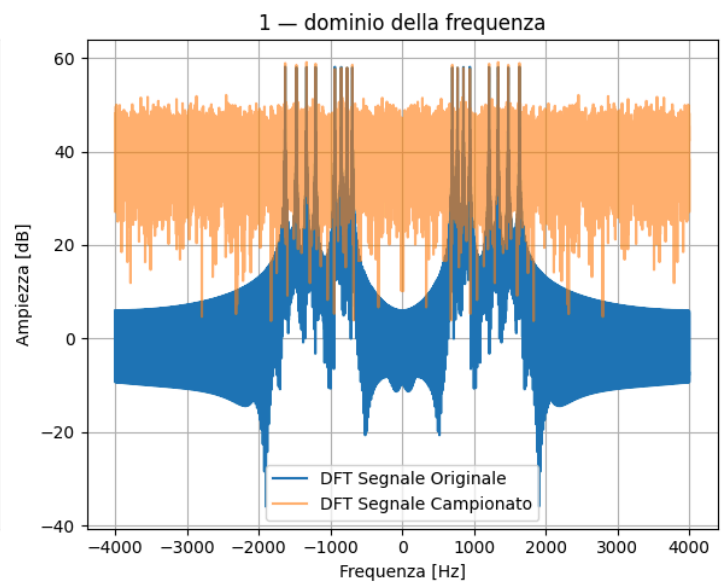
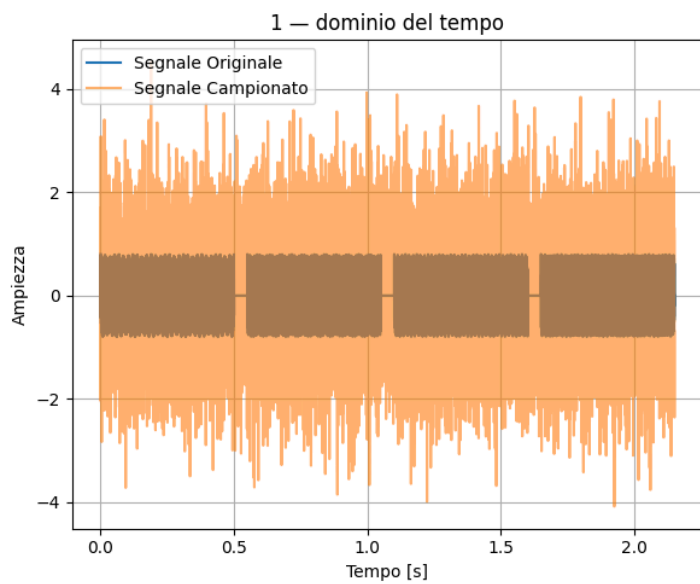
File WAV salvato in: ./esperimenti/rumore\dtmf_sequence_24Chash_500ms_fs8000.wav



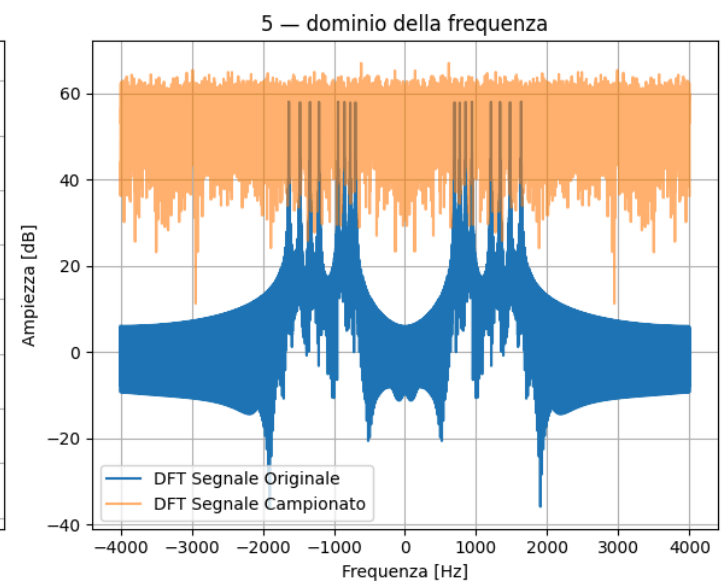
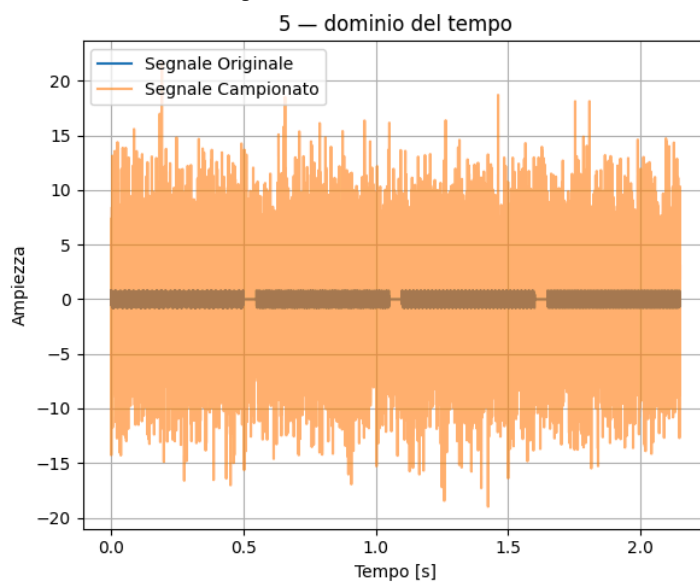
Decodifica senza denoising: 24C#
Decodifica con denoising: 24C#



Decodifica senza denoising: 24C#
Decodifica con denoising: 24C#



Decodifica senza denoising: 24C#
Decodifica con denoising: 24C#



Decodifica senza denoising: 2680
Decodifica con denoising: 24C0

MANCA COMMENTO

6- Variazione della frequenza di campionamento

```
In [16]: # 4- Variazione della frequenza di campionamento
# Funzione per plotare il segnale nel dominio del tempo e nello spettro (affiancati)
def plot_signals(t: np.ndarray, x_clean: np.ndarray, x_resampled: np.ndarray, fs: int, title_prefix: str):
    """Plotta segnale e segnale campionato affiancati (dominio del tempo e della frequenza)."""
    plt.figure(figsize=(12, 5))
```

```

# Plot 1: Tempo - Segnale e Segnale Campionato
plt.subplot(1, 2, 1)
plt.plot(t, x_clean, label="Segnale Originale", color='tab:blue') # Colore blu per il segnale originale
plt.plot(np.arange(len(x_resampled)) / fs, x_resampled, label="Segnale Campionato", color='tab:orange', alpha=0.6) # Colore arancione co
plt.title(f"{title_prefix} - dominio del tempo")
plt.xlabel("Tempo [s]")
plt.ylabel("Ampiezza")
plt.legend()
plt.grid(True)

# Plot 2: Frequenza - DFT del segnale originale e campionato
freqs_clean, mag_clean = compute_spectrum(x_clean, fs)
freqs_resampled, mag_resampled = compute_spectrum(x_resampled, fs)

plt.subplot(1, 2, 2)
plt.plot(freqs_clean, 20 * np.log10(mag_clean + 1e-12), label="DFT Segnale Originale", color='tab:blue') # Colore blu per DFT del segnal
plt.plot(freqs_resampled, 20 * np.log10(mag_resampled + 1e-12), label="DFT Segnale Campionato", color='tab:orange', alpha=0.6) # Colore
plt.title(f"{title_prefix} - dominio della frequenza")
plt.xlabel("Frequenza [Hz]")
plt.ylabel("Ampiezza [dB]")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Funzione per eseguire l'esperimento con frequenze di campionamento diverse
def experiment_sampling_frequencies(sequence: str = "24C#",
                                   toneDuration: float = 0.5,
                                   fs_original: int = 8000,
                                   fs_values: List[int] = [2000, 8000, 16000, 44100], # Aggiungiamo 2000Hz per vedere aliasing
                                   noise_std: float = 0.05, # Aggiungiamo rumore
                                   out_dir: str = "./esperimenti/frequenze_campionamento"):
    """
    Genera la sequenza, la campiona a diverse frequenze e valuta l'accuratezza.
    Ritorna risultati aggregati per frequenza di campionamento.
    """
    os.makedirs(out_dir, exist_ok=True)
    x_clean, fs, _ = dialNumber(sequence, toneDuration, fs=fs_original, interToneGap=0.05, amplitude=0.8, out_dir=out_dir)
    t = np.arange(len(x_clean)) / fs

    # Aggiungiamo rumore gaussiano
    noise = np.random.normal(0, noise_std, size=x_clean.shape)
    x_noisy = x_clean + noise

    # Fase 1: Senza applicare alcun filtro (Aliasing)
    for fs_resample in fs_values:
        print(f"\nFase 1 - Senza Filtro: Campionamento a {fs_resample} Hz")

        # Resampling del segnale (senza anti-aliasing)
        x_resampled_no_filter = resample(x_noisy, int(len(x_noisy) * fs_resample / fs))

        # Plot dei segnali (tempo e frequenza)
        plot_signals(t, x_clean, x_resampled_no_filter, fs_resample, f"Senza Filtro - {fs_resample} Hz")

        # Decodifica del segnale campionato (senza filtro)
        decoded, details = decode_dtmf(x_resampled_no_filter, fs_resample, toneDuration=toneDuration, segmentation='fixed', zero_pad_factor=1)
        print(f"Decodifica senza denoising a {fs_resample} Hz: {decoded}")

    # Fase 2: Con applicazione di filtro anti-aliasing
    for fs_resample in fs_values:
        print(f"\nFase 2 - Con Filtro Anti-Aliasing: Campionamento a {fs_resample} Hz")

        # Filtraggio anti-aliasing prima del downsampling
        cutoff = min(fs_resample / 2, 3000) # Frequenza di taglio per evitare aliasing
        x_filtered = lowpass_filter(x_noisy, cutoff, fs, order=6)

        # Resampling del segnale (con filtro anti-aliasing)
        x_resampled_with_filter = resample(x_filtered, int(len(x_noisy) * fs_resample / fs))

        # Plot dei segnali (tempo e frequenza)
        plot_signals(t, x_clean, x_resampled_with_filter, fs_resample, f"Con Filtro - {fs_resample} Hz")

        # Decodifica del segnale campionato (con filtro)
        decoded_filtered, details_filtered = decode_dtmf(x_resampled_with_filter, fs_resample, toneDuration=toneDuration, segmentation='fixed')
        print(f"Decodifica con denoising a {fs_resample} Hz: {decoded_filtered}")

    return _

```

```

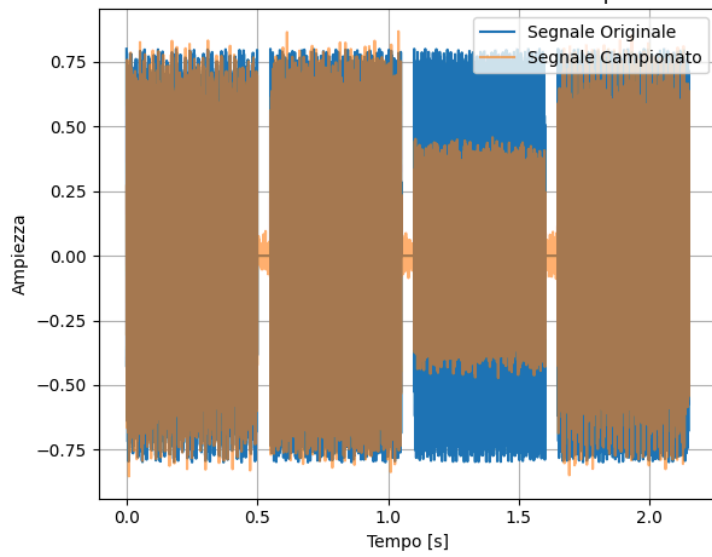
In [50]: # Esecuzione dell'esperimento con diverse frequenze di campionamento
res_sampling = experiment_sampling_frequencies(sequence="24C#", toneDuration=0.5, fs_original=8000, fs_values=[3000, 8000, 16000, 44100], noi

```

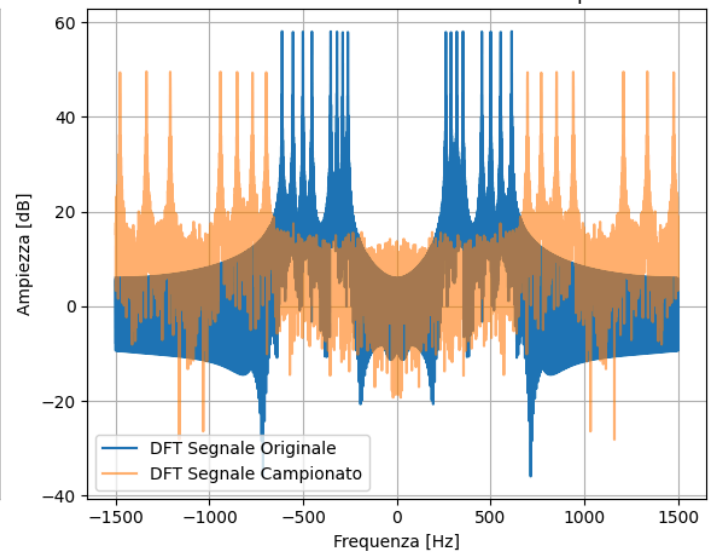
File WAV salvato in: ./esperimenti/frequenze_campionamento/dtmf_sequence_24Chash_500ms_fs8000.wav

Fase 1 - Senza Filtro: Campionamento a 3000 Hz

Senza Filtro - 3000 Hz — dominio del tempo



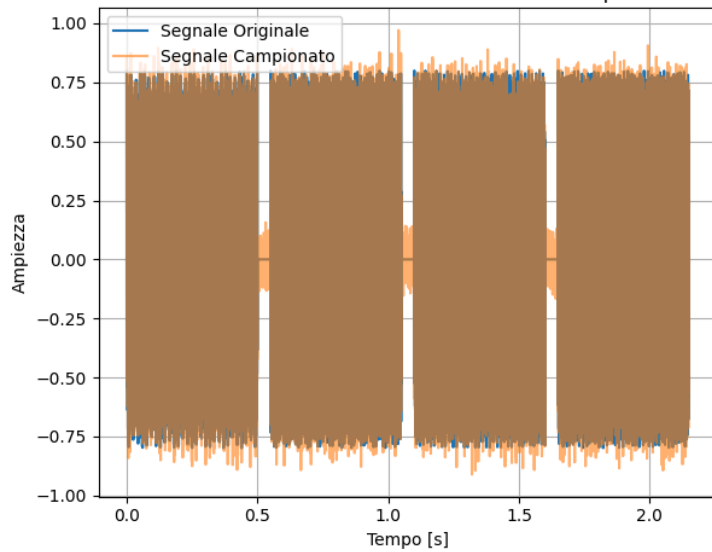
Senza Filtro - 3000 Hz — dominio della frequenza



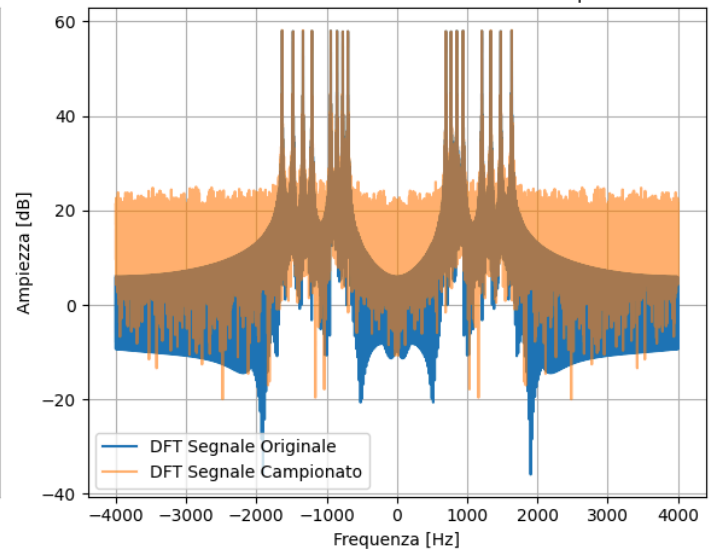
Decodifica senza denoising a 3000 Hz: 248#

Fase 1 - Senza Filtro: Campionamento a 8000 Hz

Senza Filtro - 8000 Hz — dominio del tempo



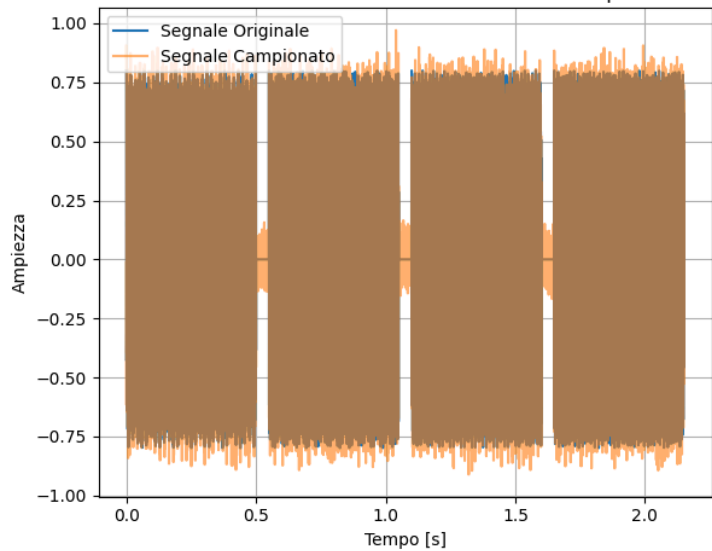
Senza Filtro - 8000 Hz — dominio della frequenza



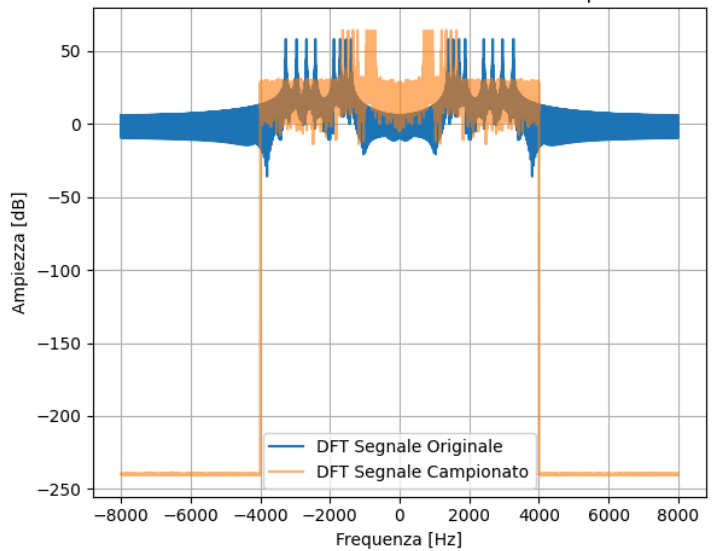
Decodifica senza denoising a 8000 Hz: 24C#

Fase 1 - Senza Filtro: Campionamento a 16000 Hz

Senza Filtro - 16000 Hz — dominio del tempo



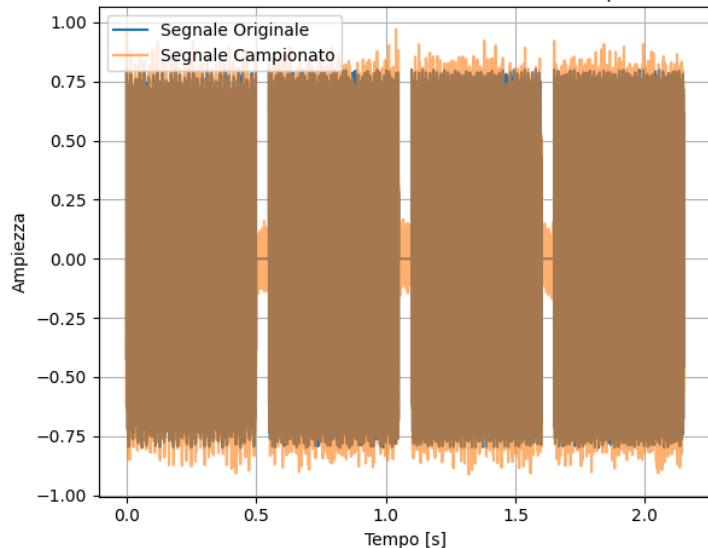
Senza Filtro - 16000 Hz — dominio della frequenza



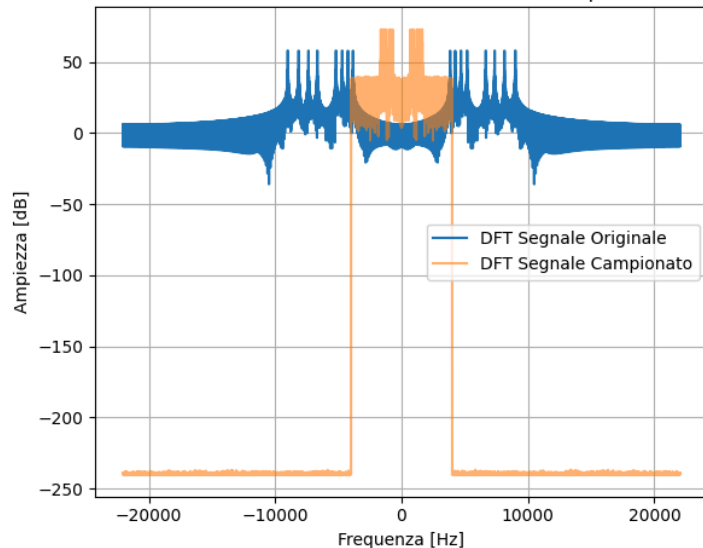
Decodifica senza denoising a 16000 Hz: 24C#

Fase 1 - Senza Filtro: Campionamento a 44100 Hz

Senza Filtro - 44100 Hz — dominio del tempo



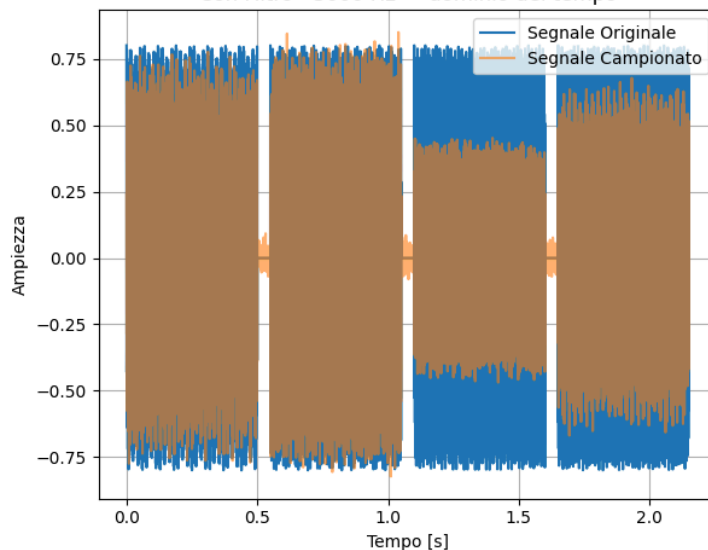
Senza Filtro - 44100 Hz — dominio della frequenza



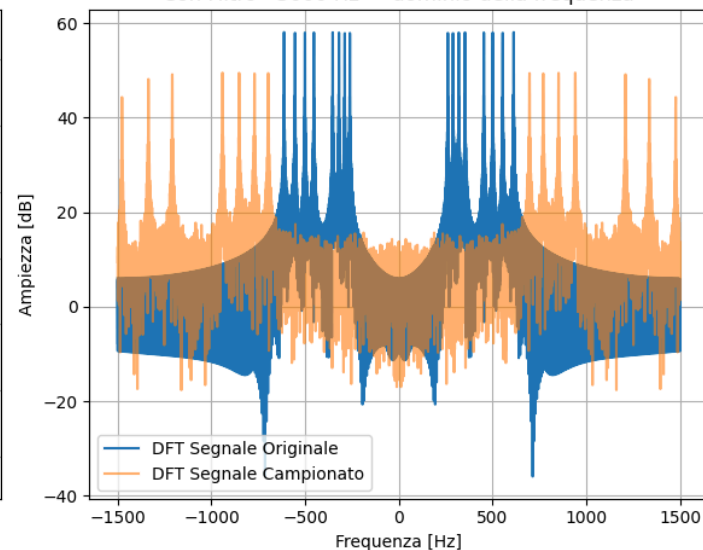
Decodifica senza denoising a 44100 Hz: 24C#

Fase 2 - Con Filtro Anti-Aliasing: Campionamento a 3000 Hz

Con Filtro - 3000 Hz — dominio del tempo



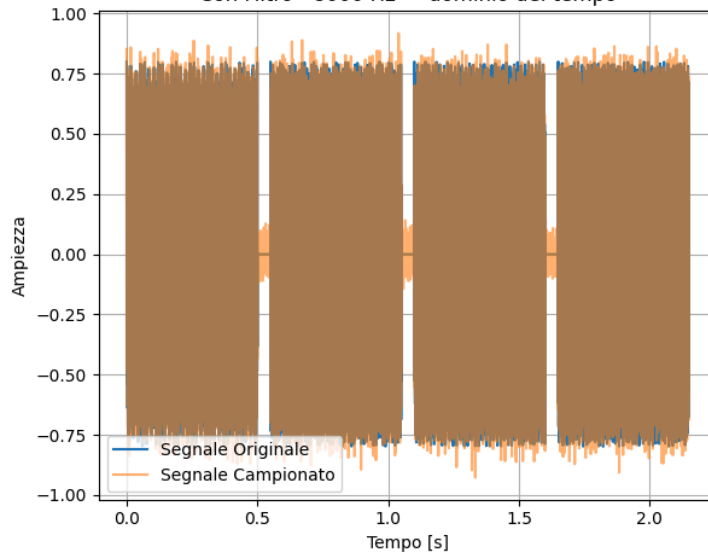
Con Filtro - 3000 Hz — dominio della frequenza



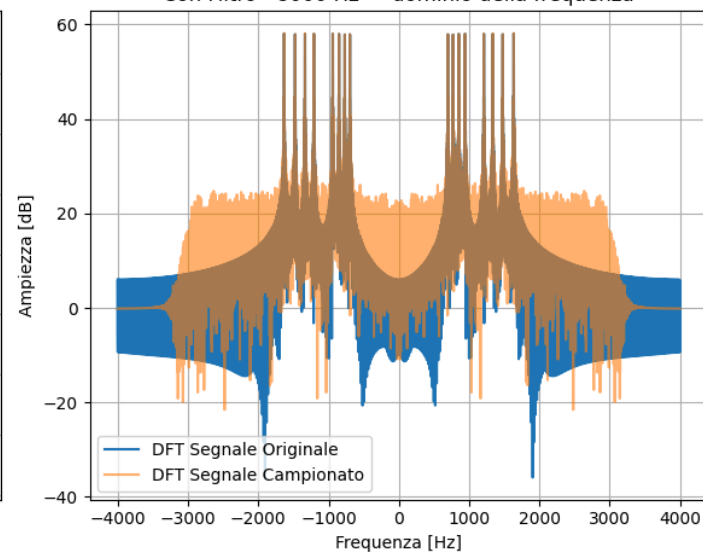
Decodifica con denoising a 3000 Hz: 248#

Fase 2 - Con Filtro Anti-Aliasing: Campionamento a 8000 Hz

Con Filtro - 8000 Hz — dominio del tempo

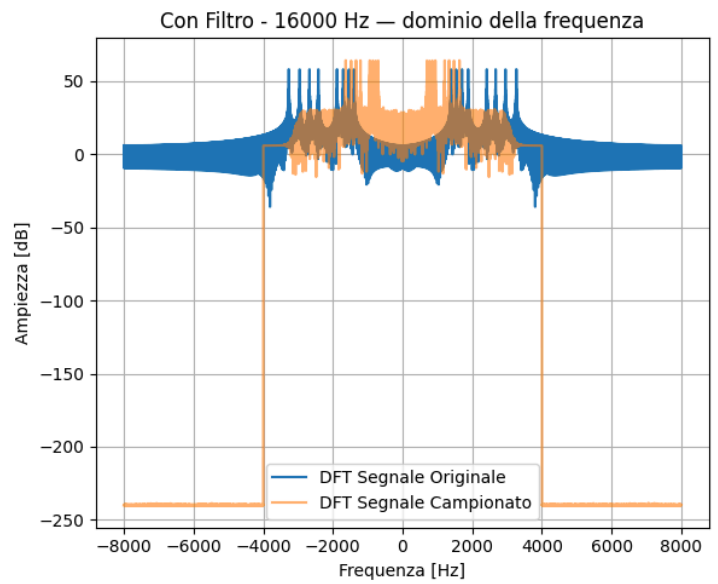
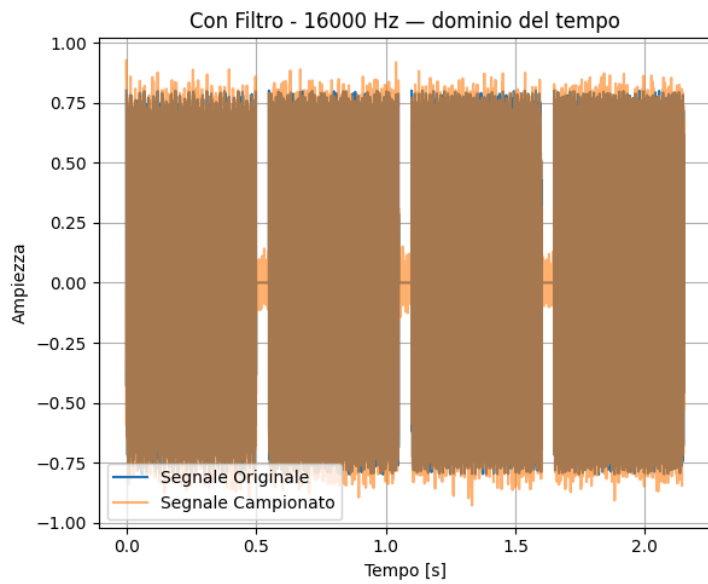


Con Filtro - 8000 Hz — dominio della frequenza



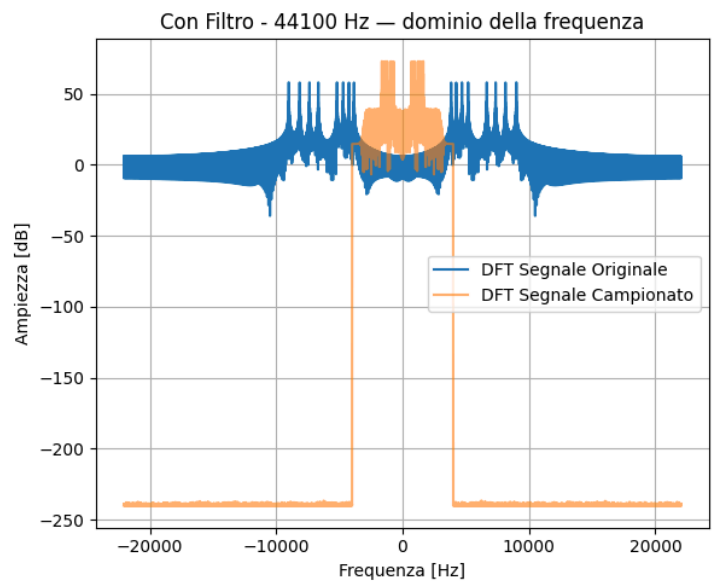
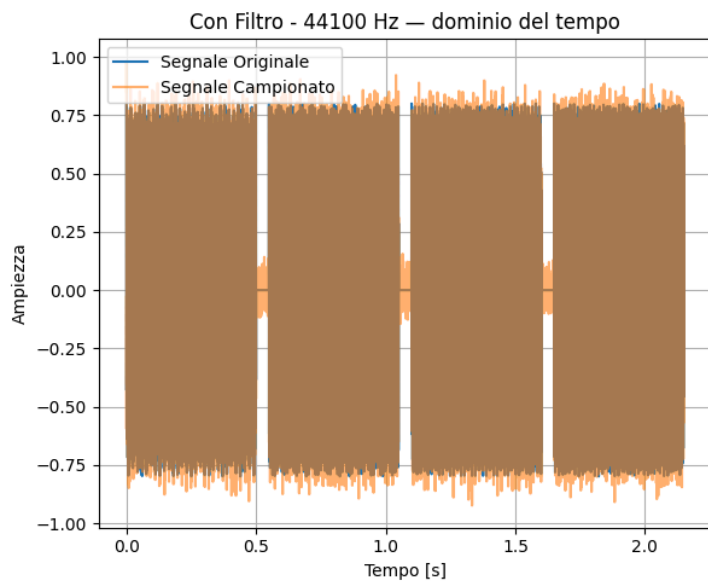
Decodifica con denoising a 8000 Hz: 24C#

Fase 2 - Con Filtro Anti-Aliasing: Campionamento a 16000 Hz



Decodifica con denoising a 16000 Hz: 24C#

Fase 2 - Con Filtro Anti-Aliasing: Campionamento a 44100 Hz



Decodifica con denoising a 44100 Hz: 24C#

MANCA COMMENTO

Conclusioni

Cosa traiamo da questo progetto?